

LMS

A Learning Management System in Django REST Framework and React

Everything about everything: the setup, every file in the codebase, every rule the server enforces, every screen the browser draws, and the reasons behind the decisions in both halves.

Contents

Part I — Orientation	4
1. What this project is	5
2. The stack, exactly	7
3. The map of the repository	9
4. One request, end to end	12
Part II — Getting it running	14
5. Prerequisites	15
6. Step 1 — The database	16
7. Step 2 — The backend	17
8. Step 3 — Seeding data	18
9. Step 4 — The frontend	19
10. Step 5 — Signing in	20
11. Step 6 — Proving it works	21
12. Every environment variable	22
Part III — The backend, file by file	24
13. <code>lms/</code> — the project package	25
13.4 <code>settings.py</code> , setting by setting	25
14. <code>models.py</code> — nine tables	29
15. <code>serializers.py</code>	32
16. <code>permissions.py</code> — the security rules	35
17. <code>filters.py</code>	38
18. <code>pagination.py</code>	40
19. <code>views.py</code>	41
20. <code>urls.py</code> — the route table	45
21. <code>admin.py</code>	46
22. <code>migrations/</code>	47
23. The seeder	48
24. <code>scripts/api_smoke_test.py</code>	51
Part IV — The frontend, file by file	53
25. Entry points and build config	54
26. <code>api.js</code> — every request	56
27. <code>auth.js</code> and <code>AuthContext.jsx</code>	59
28. <code>permissions.js</code>	61
29. <code>useTableQuery.js</code>	62
30. <code>App.jsx</code> — the routes	64
31. <code>Sidebar.jsx</code>	66
32. <code>components/</code> — the thirteen	68
33. The eight resource pages	71
34. The seven other screens	76
Part V — How it behaves	79
35. The authentication lifecycle	80
36. The list pipeline	82

37. The permission matrix	84
38. Performance, and what it cost	85
Part VI — Working on it	87
39. Recipe — add a field	88
40. Recipe — add a filter	90
41. Recipe — add a whole resource	92
42. Testing, linting, screenshots	95
43. Troubleshooting	97
44. Before it faces the real world	98
45. Rough edges and known gaps	99
Appendices	101
A. API reference	101
B. File index	104
C. Glossary	107

PART ONE

Orientation

What the project is, what it is built from, where everything lives, and what actually happens when you click something.

1. What this project is

Two applications that talk to each other over JSON, plus one database behind them.

The **backend** is a Django 5.2 project with Django REST Framework on top. It owns the database, issues the JWT tokens, and holds every rule about who is allowed to change what. The **frontend** is a React 19 single-page application built by Vite and styled with Tailwind. It has fifteen screens and no mock data anywhere: if the API is down, the pages say so rather than inventing rows.

The domain is a small online-course provider. Nine tables model it: a `Profile` beside each Django `User`, then `Teacher`, `Student`, `Course`, `Lesson`, `Assignment`, `Enrollment`, `Submission` and `Results`. Eight of those get a full CRUD screen in the browser.

1.1 The two things that surprise people first

You sign in with a phone number, not a username. The login view looks the caller up by `Profile.phone`, which is a unique column, and then checks the password against the `User` it hangs off. The username still exists — it is how an admin finds you — but it is not a credential.

There is no public sign-up. `POST /api/register/` is guarded by `IsAdmin`, which refuses every method including `GET`. An admin creates accounts from the Accounts screen inside the app and picks the role at the same time. Anonymous callers get 401; signed-in teachers and students get 403.

1.2 The three rules the codebase sticks to

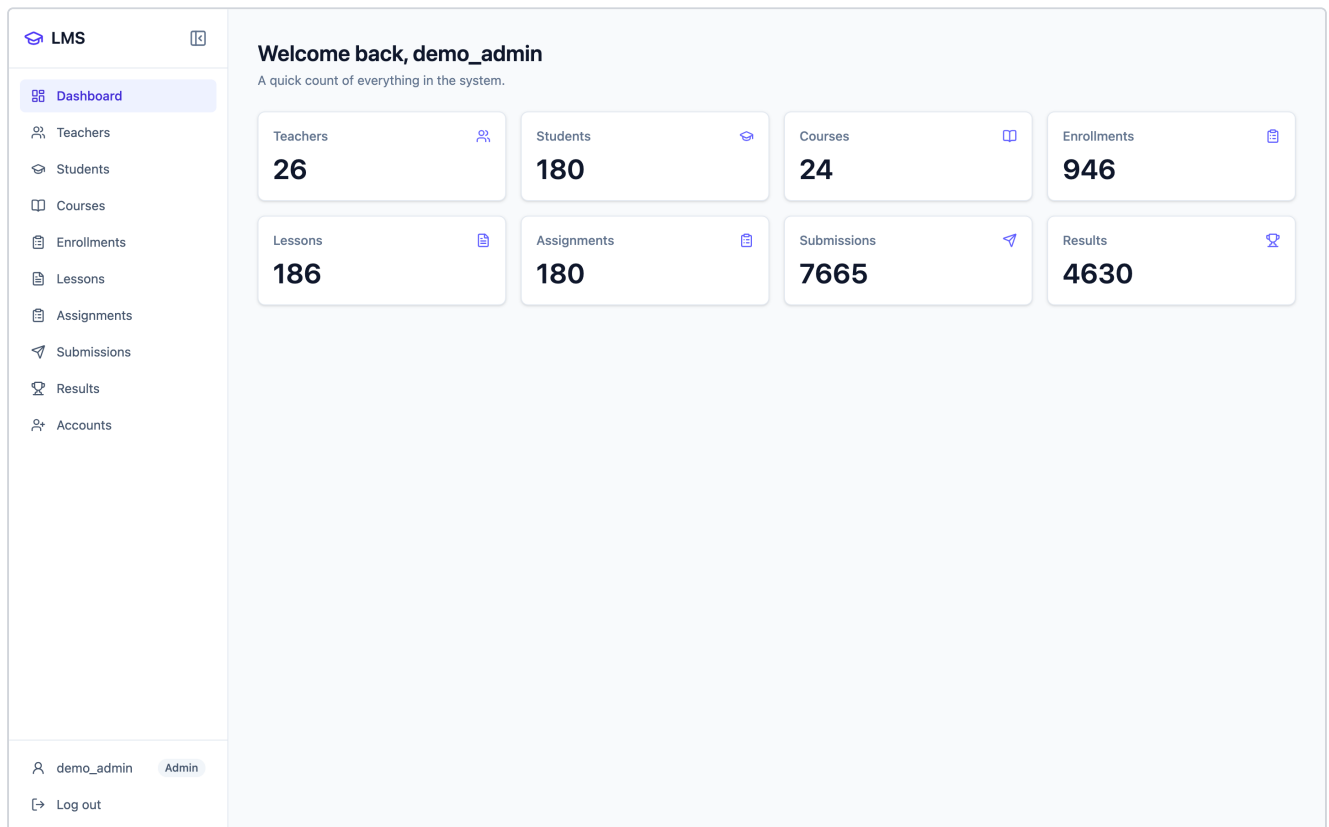
1. **The API is what enforces permissions.** The frontend hides buttons that would not work, but that is a courtesy to the reader, not a control. Every write is checked again on the server by a permission class.
2. **Every request the frontend makes lives in `src/api.js`.** Nothing else in the React app calls `fetch`. One file to read to know the whole surface the browser touches.
3. **The eight table pages are the same page eight times.** Same hook, same components, different columns and different field names. Read `pages/Students.jsx` once and you have read all eight.

1.3 What is actually in the database right now

The seeded development database on the machine this guidebook was written on holds:

Table	Rows	Table	Rows
User	212	Assignment	180
Profile	212	Enrollment	946
Teacher	26	Submission	7,665
Student	180	Results	4,630
Course	24	Lesson	186

Those numbers matter to how the code is written. Seven thousand submissions is enough that "send the whole table to the browser and let it sort things out" stops working, which is why every list endpoint is paginated and filtered on the server. See [chapter 38](#).



The dashboard: eight tiles, each a live count read from the API.

2. The stack, exactly

2.1 Backend dependencies

All six, from `backend/requirements.txt` :

Package	Pinned	What it does here
Django	5.2.7	ORM, migrations, auth, admin site, the dev server
djangorestframework	3.15.2	Serializers, generic views, the filter and pagination machinery
djangorestframework-simplejwt	5.4.0	Access and refresh tokens; its <code>token_blacklist</code> app is installed too
django-cors-headers	4.9.0	Lets the browser on :5180 call the API on :8001
mysqlclient	2.2.7	The MySQL/MariaDB driver. Needs client headers at install time
PyJWT	2.10.1	Pulled in by SimpleJWT; not imported directly anywhere

NOTE

The migration files were generated on Django 6.0.6 and the project now runs on 5.2.7. That is deliberate and handled — see `DEFAULT_AUTO_FIELD` in [chapter 13](#) — but it is why the migration headers name a version newer than the one in `requirements.txt` .

2.2 Frontend dependencies

From `frontend/package.json` . Six runtime dependencies, nine development ones.

Package	Range	What it does here
react , react-dom	^19.2.7	The UI library
react-router-dom	^7.11.0	Client-side routing, nested layout routes, <code>useSearchParams</code>
tailwindcss	^4.3.3	All styling. Tailwind 4, so there is no <code>tailwind.config.js</code>
@tailwindcss/vite	^4.3.3	The Tailwind 4 Vite plugin; replaces the old PostCSS setup
lucide-react	^1.27.0	Every icon in the app
vite	^8.1.1	Dev server and bundler
eslint + plugins	^10.6.0	Linting, flat config

There is no state library, no data-fetching library, no component library and no test runner. The app's state is `useState` and one context; its data fetching is `fetch` wrapped in `request()` ; its components are thirteen files in `src/components/` .

2.3 Versions this was tested against

Tool	Minimum	Tested on
Python	3.11	3.14
Node	20	25
MySQL / MariaDB	8.0 / 10.6	MariaDB on port 3306

3. The map of the repository

Every tracked file, with a note on what it is for. Nothing here is generated except the screenshots and this guidebook.

lms/	
├─ README.md	the short version of this book
├─ .gitignore	__pycache__, venv/, .env, node_modules/, .DS_Store
├─ backend/	the Django project root (manage.py lives here)
│ ├── manage.py	unmodified startproject boilerplate
│ └─ requirements.txt	six pinned packages
│ └─ lms/	the project package – settings and root routing
│ ├── settings.py	everything configurable, plus long comments
│ ├── urls.py	/admin/ and /api/ only
│ ├── wsgi.py asgi.py	deployment entry points, untouched
│ └─ __init__.py	
│ └─ backend/	the one Django app. All the code is here.
│ ├── models.py	9 models, ~100 lines
│ ├── serializers.py	11 serializers: auth, profile, and 8 resources
│ ├── views.py	6 auth views + 16 resource views
│ ├── urls.py	22 routes under /api/
│ ├── permissions.py	role_of() and 5 permission classes ← the rules
│ ├── filters.py	FieldFilterBackend: ?field= and ?from=?to=
│ ├── pagination.py	StandardPagination: the list envelope
│ ├── admin.py	9 ModelAdmins; role is editable inline
│ ├── apps.py	BackendConfig, nothing custom
│ ├── tests.py	empty – see chapter 45
│ ├── seed_catalog.py	470 lines of names, courses, briefs, feedback
│ └─ migrations/	
│ ├── 0001_initial.py	Student, Teacher, Profile
│ ├── 0002_course_enrollment_...py	the other six models
│ ├── 0003_profile_role.py	adds Profile.role
│ └─ 0004_existing_accounts_become_admins.py	a data migration
│ └─ management/commands/	
│ └─ seed_data.py	488 lines. Idempotent. See chapter 23.
└─ scripts/	
└─ api_smoke_test.py	487 lines. Drives every endpoint over real HTTP.
├─ frontend/	
│ ├── index.html	one div, one module script
│ ├── package.json	package-lock.json
│ ├── vite.config.js	React + Tailwind plugins, port 5180
│ ├── eslint.config.js	flat config, ESLint 10
│ ├── .env.example	VITE_API_URL, and why it ends in /api
│ ├── .gitignore	node_modules, dist, *.local
│ ├── README.md	the frontend-only quickstart
│ ├── public/favicon.svg	
│ └─ src/	
│ ├── main.jsx	BrowserRouter > AuthProvider > App
│ ├── App.jsx	every route, and which ones are protected
│ ├── index.css	one line: @import "tailwindcss"
│ ├── api.js	every call to the backend, and the session
│ ├── auth.js	the context object and useAuth()
│ ├── AuthContext.jsx	the provider: log in, log out, top up the role
│ ├── permissions.js	a mirror of the backend rules, for buttons only
│ ├── useTableQuery.js	page/search/sort/filter state, written once
│ ├── Sidebar.jsx	the menu and the frame around signed-in pages
│ ├── components/	13 files: Table, Pagination, FilterBar, Alert...
│ └─ pages/	15 files, one per screen
└─ docs/	
├── capture_screenshots.py	Playwright script that redraws the images
├── screenshots/	18 PNGs used by the README and by this book
├── guidebook.html	the source of this document
└─ LMS-Guidebook.pdf	this document

3.1 The one confusing name

The Django *project* package is `backend/lms/` and the Django *app* is `backend/backend/`. So the import path for the app's models is `backend.models`, and the settings module is `lms.settings`. Both live under the top-

level `backend/` directory, which is a third, unrelated use of the word. When this book says "the app", it means `backend/backend/`.

3.2 Where to start reading

In this order, and nothing else is needed to understand the shape of the whole thing:

1. `backend/backend/models.py` — what exists
2. `backend/backend/permissions.py` — who may change it
3. `backend/backend/views.py` — how it is exposed
4. `frontend/src/api.js` — how the browser asks
5. `frontend/src/pages/Students.jsx` — the shape of all eight table pages

4. One request, end to end

Follow a single click — typing "nusrat" into the search box on the Students page — through every file it touches.

browser	server
<pre> 1 keystroke in <FilterBar> onSearchChange 2 useTableQuery: setSearchBox("nusrat") 300 ms pause (debounce) 3 setSearch("nusrat"), setPage(1) params useMemo rebuilds 4 Students.jsx effect re-runs studentsApi.list(params) 5 api.js toQueryString drops the empties GET /api/student/?page=1&page_size=10&search=nusrat&ordering=id Authorization: Bearer eyJhbGciOi...</pre>	<pre> 6 CorsMiddleware: origin allowed? 7 JWTAuthentication: token valid? → request.user 8 AdminWrites.has_permission: GET is safe → True 9 SearchFilter: name/email/roll_number icontains 10 FieldFilterBackend: no filter params present 11 OrderingFilter: ?ordering=id 12 StandardPagination: LIMIT 10 OFFSET 0 + COUNT(*) 13 StudentSerializer: 10 rows → JSON</pre>
<pre> 14 request(): 200, parse body 15 setStudents(body.results); setCount(body.count); setTotalPages(body.total_pages) 16 <Table> redraws 10 rows, <Pagination> redraws the footer</pre>	

4.1 The same trace as file references

Step	File	What it does
1	<code>components/FilterBar.jsx</code>	Controlled <code><input type="search"></code> , calls <code>onSearchChange</code>
2–3	<code>useTableQuery.js</code>	Holds <code>searchBox</code> and <code>search</code> separately; a <code>setTimeout</code> moves one to the other
4	<code>pages/Students.jsx</code>	Effect keyed on <code>queryParams</code> ; sets an <code>isCurrent</code> flag to ignore stale answers
5	<code>api.js</code>	<code>resource("student").list()</code> → <code>request()</code> → <code>fetch</code> with the bearer header
6	<code>corsheaders</code>	Middleware, second in the list, before <code>CommonMiddleware</code>
7	<code>SimpleJWT</code>	<code>DEFAULT_AUTHENTICATION_CLASSES</code> ; a bad token is 401 here and never reaches the view
8	<code>permissions.py</code>	<code>RoleWritePermission</code> lets any signed-in caller read
9–11	<code>DRF + filters.py</code>	Three backends run in order, each a no-op unless the view declares its fields
12	<code>pagination.py</code>	Slices the queryset and builds the <code>{count, page, ...}</code> envelope
13	<code>serializers.py</code>	<code>StudentSerializer</code> , six fields
14–16	<code>api.js</code> , the page, <code>components/</code>	Parse, set state, re-render

TIP

Every list request in this app follows exactly that path. The only things that vary are the view's `search_fields` / `filter_fields` / `ordering_fields` and which serializer runs at step 13.

PART TWO

Getting it running

Six steps from a clean checkout to a browser showing real data, then the full list of things you can configure.

5. Prerequisites

What	Version	Why
Python	3.11 or newer	Django 5.2 needs 3.10+; this was tested on 3.14
Node.js	20 or newer	Vite 8 requires it
MySQL or MariaDB	8.0 / 10.6+	The only database backend configured
MySQL client headers	—	<code>mysqlclient</code> compiles against them at install time

On macOS with Homebrew:

```
SHELL
brew install mariadb pkg-config
brew services start mariadb
```

On Debian or Ubuntu:

```
SHELL
sudo apt install mariadb-server libmysqlclient-dev pkg-config python3-venv
```

WARNING

If `pip install mysqlclient` fails with a compiler error about `mysql.h`, the client headers are missing — not Python's fault. Install `mysql-client` (macOS) or `libmysqlclient-dev` (Debian) and retry.

6. Step 1 — The database

Create the database and a user for it. These three names are the defaults compiled into `backend/lms/settings.py`, so matching them means you can skip the environment variables entirely.

SQL

```
CREATE DATABASE lms_db CHARACTER SET utf8mb4;
CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';
GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';
```

In one line from a shell:

SHELL

```
mysql -u root -e "
CREATE DATABASE lms_db CHARACTER SET utf8mb4;
CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';
GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';
"
```

NOTE

`127.0.0.1` and `localhost` are different hosts to MySQL's grant system: one means TCP, the other means the unix socket. The settings file uses `127.0.0.1`, so grant to that. Granting to `'lms_user'@'localhost'` instead is the most common reason for `Access denied` on the first `migrate`.

7. Step 2 — The backend

SHELL

```
cd backend
python -m venv venv && source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

python manage.py migrate
python manage.py runserver 8001
```

Four migrations run, in order: the three original models, then the other six, then `Profile.role`, then the data migration that promotes any pre-existing account to admin. On an empty database that last one updates nothing.

Two addresses are now live:

Address	What
<code>http://127.0.0.1:8001/api/</code>	The JSON API. A browser hitting it gets DRF's browsable pages.
<code>http://127.0.0.1:8001/admin/</code>	Django admin. Nine models registered; this is where roles are changed.

WARNING

Port 8001, not 8000. The frontend's default `VITE_API_URL`, the smoke test's default `LMS_API`, and the CORS list all assume 8001. Running `runserver` with no argument gives you 8000 and nothing will connect.

7.1 If there is no admin account yet

SHELL

```
python manage.py createsuperuser
```

A superuser has no `Profile` row, and therefore no `role` column to read. `role_of()` in `permissions.py` checks `user.is_superuser` first and returns `admin`, so `createsuperuser` is always a way back into a locked-out system.

8. Step 3 — Seeding data

```
SHELL
python manage.py seed_data
```

This fills the database with a realistic amount of realistic-looking data — around 180 students, two dozen courses, several thousand submissions and a few thousand grades — and guarantees the three demo logins. It takes a few seconds and prints a table of totals when it finishes.

8.1 The three options

Flag	Default	Effect
<code>--teachers N</code>	26	How many <code>Teacher</code> rows to <i>end up with</i> , not how many to add
<code>--students N</code>	180	Same, for <code>Student</code>
<code>--fresh</code>	off	Delete every seeded row first, then rebuild
<code>--seed N</code>	20260902	Random seed for the purely cosmetic parts

```
SHELL
python manage.py seed_data           # top up whatever is missing
python manage.py seed_data --fresh   # wipe seeded rows first
python manage.py seed_data --students 400 # a bigger cohort
```

8.2 Why running it twice is safe

Every "does this row exist yet" decision is derived from the ids involved rather than drawn from a random stream. A helper called `dice()` hashes the inputs with `zlib.crc32` and takes the result modulo 100, so the same question gives the same answer in this process and in the next one. A student who had handed in four of six assignments still has four of six after a second run. Rows are found by a stable key first — a person by email, a course by title, a lesson by title within its course — so nothing is duplicated either. Full detail in [chapter 23](#).

WARNING

`--fresh` deletes *all* course data — every teacher, student, course, lesson, assignment, enrollment, submission and result in the database, seeded or not — plus every non-superuser account whose email ends in `@os-tad.example.com` or `@student.example.com`. Superusers survive. Do not point it at anything you care about.

9. Step 4 — The frontend

```
SHELL
```

```
cd frontend  
npm install  
npm run dev
```

Vite serves on **http://localhost:5180**. That port is set explicitly in `frontend/vite.config.js` because 5173–5175, Vite's own defaults, were already taken by other projects on the author's machine. If 5180 is busy too, Vite quietly takes the next free port and prints which one.

WARNING

If Vite moves to another port, add that origin to `CORS_ALLOWED_ORIGINS` in `backend/lms/settings.py` or every request will be blocked by the browser before it leaves. The list currently allows 5180 and 5173, on both `localhost` and `127.0.0.1`.

9.1 Pointing it at a different backend

```
SHELL
```

```
cp .env.example .env.local  
# then edit:  
VITE_API_URL=http://127.0.0.1:8001/api
```

Two things about that variable. It must start with `VITE_` — Vite only exposes those to the browser bundle. And it must end in `/api`, because every path in `src/api.js` is written relative to it (`"/login/"`, `"/student/"`, and so on). `.env.local` is gitignored, which is what makes it the right place for machine-specific values.

10. Step 5 — Signing in

Open <http://localhost:5180>. Any route redirects to `/login` without a token.

Role	Phone	Password	What it can do
Admin	01700000000	admin1234	Everything, including creating accounts
Teacher	01700000001	teacher1234	Course material, enrollments, grading
Student	01700000002	student1234	Read everything, hand in a submission

The seeder repairs these three on every run — it sets the phone and the password unconditionally, and even moves another account off the number if one is squatting on it — so the login page's one-click buttons always work.

The login page in `npm run dev`. The dashed amber block is wrapped in `import.meta.env.DEV`, so Vite drops it from a production build.

WARNING

These are development fixtures with published passwords. They exist in whatever database you point the seeder at. Never run `seed_data` against anything real.

11. Step 6 — Proving it works

With the server running in one terminal:

```
SHELL
cd backend
python scripts/api_smoke_test.py
```

This is not a unit test suite. It talks HTTP to a real server with a real database, which is the only way to catch the class of problem that only appears there: a permission class wired to the wrong view, a serializer that rejects its own output, a route that was never added to `urls.py`.

Every check states the status code it expects, and **a check that expects 403 passes only on 403**. The permission rules are part of what "working" means, so a create that succeeds when it should have been refused is a failure, not a bonus. Anything the script creates it deletes on the way out — except the one account the registration check makes, because the API has no delete-user route; it says so and tells you to remove it from `/admin/`.

```
Target: http://127.0.0.1:8001/api

Authentication
-----
pass log in as admin [200]
pass admin gets both tokens and the right role
...
Writing (create, update, delete, and who may)
-----
pass anonymous cannot create a teacher [401]
pass teacher cannot create a teacher [403]
pass student cannot create a teacher [403]
pass admin creates a teacher [201]
...
=====
All 214 checks passed.
```

It exits 0 on success, 1 on any failure, and 2 if it cannot reach the server or sign in. Point it elsewhere with `LMS_API=http://host:port/api`.

12. Every environment variable

Nothing here is required — every one has a default in the settings file — but this is the complete list of what can be overridden.

12.1 Backend

Variable	Default	Used for
DB_NAME	lms_db	The MySQL connection in <code>DATA-BASES['default']</code>
DB_USER	lms_user	
DB_PASSWORD	lms_user_pw	
DB_HOST	127.0.0.1	
DB_PORT	3306	
FRONTEND_PASSWORD_RESET_URL	http://localhost:5180/reset-password	Where the emailed reset link points
EMAIL_BACKEND	...console.EmailBackend	Console prints mail instead of sending it
DEFAULT_FROM_EMAIL	no-reply@example.com	The From: on reset mail
EMAIL_HOST	smtp.gmail.com	SMTP, only used if you switch the backend to <code>smtp</code>
EMAIL_PORT	587	
EMAIL_HOST_USER	a hardcoded gmail address	
EMAIL_HOST_PASSWORD	empty	
EMAIL_USE_TLS / EMAIL_USE_SSL	True / False	

ROUGH EDGE

`SECRET_KEY`, `DEBUG` and `ALLOWED_HOSTS` are *not* read from the environment — they are literals in the settings file, with the development values committed. That is fine for development and is the first thing to change otherwise. See [chapter 44](#).

12.2 Frontend

Variable	Default	Used for
VITE_API_URL	http://127.0.0.1:8001/api	The base every path in <code>api.js</code> hangs off

Read at build time by Vite via `import.meta.env`, with the fallback written into `api.js` itself:

```
JAVASCRIPT
const BASE_URL = import.meta.env.VITE_API_URL ?? "http://127.0.0.1:8001/api";
```

12.3 Smoke test

Variable	Default
LMS_API	http://127.0.0.1:8001/api

PART THREE

The backend, file by file

Twelve files hold the entire server. This part goes through all of them in dependency order: configuration, then data, then rules, then the machinery that turns a query string into a page of rows, then the views that tie it together.

13. `lms/` — the project package

13.1 `manage.py`

Unmodified `startproject` boilerplate. It sets `DJANGO_SETTINGS_MODULE=lms.settings` and hands off to `execute_from_command_line`. Worth knowing only because every command in this book is run from `backend/`, where this file lives.

13.2 `lms/urls.py`

```
PYTHON
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

Two lines. Everything else is in the app's own `urls.py` ([chapter 20](#)). There is no route serving the React build — in development Vite serves the frontend itself, and nothing in the repository wires up a production static build.

13.3 `wsgi.py` and `asgi.py`

Both untouched. `WSGI_APPLICATION = 'lms.wsgi.application'` in settings points at the first; nothing points at the second, but it is there if you want to run under an ASGI server.

13.4 `settings.py`, setting by setting

This file is longer than a generated one because most decisions in it carry a comment explaining why. Here is every block.

Security and debug

```
PYTHON
SECRET_KEY = 'django-insecure-@hg+%( *w2m0jh=8$ksnpsoe+dv2jb+py3@1b(3vhohjb7cx8!'
DEBUG = True
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
```

Development values, committed. The key is the one `startproject` generated; the `django-insecure-` prefix is Django's own marker that it was machine-generated and never rotated.

`INSTALLED_APPS`

The six Django contrib apps, then four more:

App	Why it is there
backend	The one app in the project
rest_framework	DRF itself
rest_framework_simplejwt.token_blacklist	Gives the two tables <code>blacklist_user_tokens()</code> writes to when a password changes
corsheaders	Cross-origin headers for the Vite dev server

MIDDLEWARE

Django's standard seven, with `corsheaders.middleware.CorsMiddleware` inserted second — after `SecurityMiddleware` and before `CommonMiddleware`. That position matters: CORS headers have to be attached before `CommonMiddleware` can issue a redirect that would strip them.

CORS_ALLOWED_ORIGINS

```
PYTHON
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5180',
    'http://127.0.0.1:5180',
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

5180 is where Vite actually runs. 5173 is kept because it is Vite's own default and the port older versions of the README named. `localhost` and `127.0.0.1` are distinct origins to a browser, so both spellings are listed.

REST_FRAMEWORK

Five keys, and each one is a decision worth understanding.

Key	Value	Consequence
DEFAULT_AUTHENTICATION_CLASSES	JWTAuthentication	The only way to authenticate. No session auth on the API, so the browsable API cannot be used while logged into <code>/admin/</code> .
DEFAULT_PERMISSION_CLASSES	IsAuthenticated	Fails closed. DRF's own default is <code>AllowAny</code> ; without this line, any view added without explicit <code>permission_classes</code> would be public.
DEFAULT_FILTER_BACKENDS	SearchFilter, FieldFilterBackend, OrderingFilter	All three run on every list view, in that order. Each is a no-op unless the view declares the matching attribute, so detail routes are untouched.
DEFAULT_PAGINATION_CLASS	StandardPagination	Lists come back as an envelope, never a bare array.
PAGE_SIZE	10	The default page. Overridable per request up to 200.

SIMPLE_JWT

```
PYTHON

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=8),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

The library default access lifetime is five minutes, which forces a re-login in the middle of a task. **There is no refresh endpoint in this project** — the refresh token is issued and handed to the browser, but nothing ever exchanges it — so the access token is widened to eight hours instead. That single fact explains several decisions in the frontend: see [chapter 35](#).

DATABASES

```
PYTHON

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('DB_NAME', 'lms_db'),
        'USER': os.getenv('DB_USER', 'lms_user'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'lms_user_pw'),
        'HOST': os.getenv('DB_HOST', '127.0.0.1'),
        'PORT': os.getenv('DB_PORT', '3306'),
    }
}
```

MySQL only. There is no SQLite fallback, so the database has to exist before `migrate` will run.

Password validation

All four of Django's stock validators are on: `UserAttributeSimilarity`, `MinimumLength` (8), `CommonPassword` and `NumericPassword`. They are enforced in three places, all through `validate_password()`: account creation, self-service password change, and password reset. This is why the smoke test's "weak password refused" checks expect 400 on `"12345678"` — it is both too common and entirely numeric.

Password reset

```
PYTHON

FRONTEND_PASSWORD_RESET_URL = os.getenv(
    "FRONTEND_PASSWORD_RESET_URL", "http://localhost:5180/reset-password"
)
PASSWORD_RESET_TIMEOUT = 60 * 60 # 1 hour
```

The view appends `?uid=...&token=...` to that URL and mails the result. One hour is Django's `default_token_generator` window, checked server-side on the way back in.

Email

The default backend is `console.EmailBackend`: reset mail is printed to the terminal running `runserver`, not sent. Everything else in the block (`EMAIL_HOST`, port 587, TLS on) is a Gmail SMTP configuration waiting for you to switch `EMAIL_BACKEND` and supply an app password.

DEFAULT_AUTO_FIELD

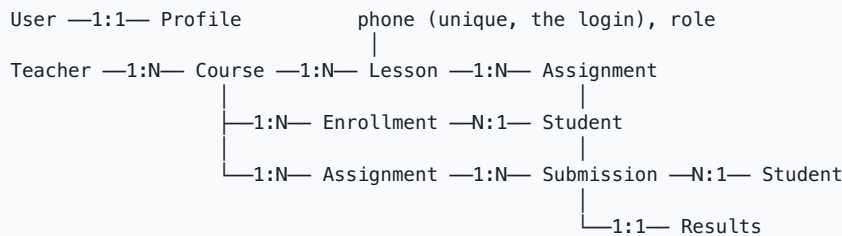
```
PYTHON
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

The migrations were generated on Django 6, where `BigAutoField` is the default primary key type. Django 5.2 falls back to `AutoField` unless told otherwise, which would make `makemigrations` want to rewrite every primary key even though the columns in the database are already `bigint`. This one line prevents a pointless migration.

14. models.py — nine tables

About a hundred lines describe the entire domain. Every model is a plain `models.Model` with no custom manager, no `Meta` class, and no `save()` override.

14.1 The shape of it



14.2 Profile

PYTHON

```

class Profile(models.Model):
    ADMIN = "admin"; TEACHER = "teacher"; STUDENT = "student"
    ROLE_CHOICES = [(ADMIN, "Admin"), (TEACHER, "Teacher"), (STUDENT, "Student")]

    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20, unique=True)
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default=STUDENT)
  
```

This is the join between Django's built-in `User` and everything this app adds. `phone` is unique because it is the credential — `LoginView` does `Profile.objects.get(phone=phone)` and works back to the user. `role` defaults to the least privileged value, so an account created by any path that forgets to set it is a student.

NOTE

Two kinds of account have no `Profile` at all: a superuser made with `createsuperuser`, and anything created outside the register endpoint. `role_of()` handles both — superuser means admin, no profile means student. Note the consequence: **a superuser has no phone number and therefore cannot log in through the app's own login page**. It can only use `/admin/`.

14.3 The eight domain models

Model	Fields	Points at
Teacher	name, email (both nullable), subject, is_active	—
Student	name, email, roll_number (all nullable), enrollment_date (required), is_active	—
Course	title, description	teacher → Teacher
Lesson	title, description	course → Course
Assignment	title, description, due_date	lesson → Lesson, course → Course
Enrollment	enrollment_date (auto_now_add)	student, course
Submission	content, submitted_at (auto_now_add)	assignment, student
Results	score (float), feedback	submission → OneToOne

Every foreign key is `on_delete=models.CASCADE`. Deleting a teacher takes their courses, which takes those courses' lessons and assignments, which takes the submissions against them, which takes the grades. The confirm dialog on the Teachers page says so in as many words: *"Their courses go too. This cannot be undone."*

14.4 The four details that shape the rest of the code

1. `Results.submission` is one-to-one

A submission can be graded exactly once; a second attempt is a 400 from the database constraint. That is what makes `?ungraded=true` possible: it maps to `results__isnull=True`, which is the database answering "nothing points back at this row yet". The Results page builds its dropdown from exactly that query ([chapter 33](#)).

2. `Assignment` points at both a course and a lesson

Redundant on paper — a lesson already knows its course — and nothing enforces that the two agree. The frontend compensates by filtering the lesson dropdown down to the chosen course's lessons, but the API would happily accept a mismatched pair.

3. Nothing links a `Student` row to a `User` login

They are two separate tables with no foreign key between them. The seeder ties them together by email as a convention, and nothing else does. This is the single most important structural fact in the project, and the reason for the odd shape of the submission permission: a student may `POST` a submission but not edit or delete one, because the server has no way to tell whose work it is.

4. Two `auto_now_add` fields

`Enrollment.enrollment_date` and `Submission.submitted_at` are set by the database on insert and cannot be supplied by the API. The seeder works around this by inserting the rows and then rewriting the dates with `bulk_update`, which is why seeded enrollments spread over two years instead of all landing on today.

14.5 `__str__`

Every model has one, and they are what the Django admin dropdowns show. A couple reach across relations — `Results.__str__` reads `self.submission.student.name` — so listing results in the admin without `select_related` is one query per row. That is only the admin; the API's `ResultsListCreateView` does select the relations.

15. serializers.py

Eleven serializers. Four are about accounts and passwords and carry all the validation; the other seven are thin wrappers over a model, plus the read-only name fields that stop the browser having to do joins.

15.1 RegisterSerializer

A `ModelSerializer` over `User` with three extra fields: `phone` (write-only, lands on the `Profile`), `first_name` and `email` made required, and `role` as a `ChoiceField` defaulting to `student`.

Validator	Rejects
<code>validate_phone</code>	A phone already on some other <code>Profile</code>
<code>validate_username</code>	A username already taken
<code>validate_password</code>	Anything Django's four password validators refuse

`create()` is wrapped in `@transaction.atomic` so a failure between making the `User` and making the `Profile` cannot leave a profile-less account behind. `to_representation()` adds the role to the response so the caller can see what was applied.

NOTE

Letting the caller choose `role` is only safe because the view above it is `IsAdmin`. If that permission class were ever relaxed, this field would become a privilege-escalation hole.

15.2 LoginSerializer

Two fields, `phone` and `password`, both required. No validation beyond presence — the view does the lookup and the password check itself, and answers with a deliberately vague "Invalid phone or password" either way so the endpoint cannot be used to find out which numbers are registered.

15.3 ProfileUpdateSerializer

A plain `Serializer`, not a `ModelSerializer`, because it writes to two tables. It accepts exactly four optional fields: `first_name`, `last_name`, `email`, `phone`.

What is deliberately *not* on the list is the interesting part:

Field	Why it is excluded
<code>role</code>	Changing your own would be a self-promotion. Admins only, from Django admin.
<code>username</code>	It is how an admin finds you, and it is not a credential, so there is nothing to gain.
<code>password</code>	Has its own endpoint, because it needs the old password first.

Because DRF ignores unknown keys, sending `{"role": "admin"}` here succeeds with a 200 and changes nothing. The smoke test checks exactly that, then re-reads the profile to confirm the role did not move.

`validate_phone` refuses a blank (it is the login credential) and refuses one that belongs to another account. `validate_email` refuses a duplicate too, with a different reason: password reset looks an account up by email, so two accounts sharing one would make that ambiguous.

15.4 `ChangePasswordSerializer`

Three write-only fields and four rules:

1. `current_password` must actually be the current one. Knowing the old password is what makes the account yours to change; without it, a borrowed browser tab would be enough to take it.
2. The two new passwords must match.
3. The new one must differ from the old one.
4. The new one must pass Django's validators, given the user for the similarity check.

15.5 The password-reset pair

`PasswordResetRequestSerializer` is one email field. `PasswordResetConfirmSerializer` does the real work in `validate()`: match the two passwords, base64-decode the `uid` into a primary key, load an active user, check the token with `default_token_generator`, then run the password validators. It stashes the resolved user in `attrs["user"]` so the view does not repeat the lookup.

Every failure path answers 400 with a field-shaped error — `uid` for a malformed link, `token` for an expired or forged one — so the frontend can show something specific.

15.6 The seven resource serializers

These are almost all `ModelSerializer` with an explicit `fields` list. The one thing they add is a **read-only name beside every id**:

Serializer	Model fields	Added read-only
<code>TeacherSerializer</code>	id, name, email, subject, is_active	—
<code>StudentSerializer</code>	id, name, email, enrollment_date, is_active, roll_number	—
<code>CourseSerializer</code>	id, title, description, teacher	<code>teacher_name</code>
<code>LessonSerializer</code>	id, title, description, course	<code>course_title</code>
<code>AssignmentSerializer</code>	id, title, description, lesson, due_date, course	<code>course_title</code> , <code>lesson_title</code>
<code>EnrollmentSerializer</code>	id, student, course, enrollment_date	<code>student_name</code> , <code>course_title</code>
<code>SubmissionSerializer</code>	id, assignment, student, submitted_at, content	<code>student_name</code> , <code>assignment_title</code>
<code>ResultSerializer</code>	id, submission, score, feedback	<code>student_name</code> , <code>assignment_title</code>

They are declared with `source`, so a single line reaches through a relation:

PYTHON

```
class ResultSerializer(serializers.ModelSerializer):
    # Two relations deep: a result points at a submission, which points at the
    # student and the assignment. "Submission #4" on its own says nothing.
    student_name = serializers.CharField(
        source='submission.student.name', read_only=True,
    )
    assignment_title = serializers.CharField(
        source='submission.assignment.title', read_only=True,
    )
```

Why the names are sent at all

The alternative was for the browser to download every teacher, every course and every student and do the join itself. That worked while lists came back whole. It stopped working once they were paginated, for two separate reasons: a page of ten rows should not cost eight thousand, and **a lookup list that is itself paginated cannot answer for a row on page 40**. The joins are already in the queryset via `select_related`, so sending the names costs nothing extra.

NOTE

Note the class name asymmetry: it is `ResultSerializer` (singular) over a model called `Results` (plural), served at `/api/results/`. The other seven are singular throughout. Nothing depends on this; it is just a thing to know before you grep.

16. permissions.py — the security rules

Ninety lines, and the single most important file in the project. Everything else is plumbing; this is the part that says no.

16.1 The rule in one table

Resource	Admin	Teacher	Student	Class
Teacher, Student records	write	read	read	AdminWrites
Course, Lesson, Assignment	write	write	read	TeachingStaffWrites
Enrollment, Results	write	write	read	TeachingStaffWrites
Submission	write	write	POST only	SubmissionWrites
Create an account	yes	no	no	IsAdmin
Own profile / password	yes	yes	yes	IsAuthenticated

Reading is open to every signed-in account. Anonymous callers can reach exactly four endpoints: login, password reset request, password reset confirm — and nothing else.

16.2 role_of(user)

PYTHON

```
def role_of(user):
    if not user or not user.is_authenticated:
        return None
    if user.is_superuser:
        return Profile.ADMIN
    profile = Profile.objects.filter(user=user).only("role").first()
    if profile is None:
        return Profile.STUDENT
    return profile.role
```

Four cases, in order, and each is a decision:

- **Anonymous** → `None`, which matches no role and so is refused everywhere.
- **Superuser** → admin, without looking at the database. `createsuperuser` is therefore always a way back into a system with no admins left.
- **No Profile row** → student. The least privilege available, rather than an exception. An account made in the shell with `User.objects.create_user()` falls here.
- **Otherwise** → whatever the column says. `.only("role")` keeps it to one small query.

NOTE

This runs on every write request, so it is one extra query per write. Reads only pay for it on `/login/` and `/profile/`, since `SAFE_METHODS` short-circuits before the lookup.

16.3 The five classes

IsAdmin

Admins only, *for every method including GET*. Used on registration, because accounts are handed out by an admin rather than claimed by whoever finds the URL. Message: "Only an admin can create accounts."

RoleWritePermission — the base

PYTHON

```
class RoleWritePermission(BasePermission):
    roles_that_may_write = ()

    def has_permission(self, request, view):
        if not request.user or not request.user.is_authenticated:
            return False
        if request.method in SAFE_METHODS:
            return True
        return role_of(request.user) in self.roles_that_may_write
```

Not authenticated → refused. GET, HEAD, OPTIONS → allowed. Anything else → the role has to be on the list. Subclasses set one tuple and one message.

AdminWrites and TeachingStaffWrites

One tuple each: (ADMIN,) and (ADMIN, TEACHER). Between them they cover seven of the eight resources.

SubmissionWrites

PYTHON

```
class SubmissionWrites(RoleWritePermission):
    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)

    def has_permission(self, request, view):
        if (request.method == "POST"
            and request.user and request.user.is_authenticated
            and role_of(request.user) == Profile.STUDENT):
            return True
        return super().has_permission(request, view)
```

The one asymmetric rule in the project. A student may hand work in and nothing else. The reason is structural, not a policy choice: **nothing links a Student row to a login account**, so on a PATCH or DELETE the server cannot tell whether the submission belongs to the caller. Handing in is the one write that is safe without that link — the worst case is a spurious row, not a tampered-with grade.

WARNING

The gap that leaves: a student can POST a submission with any student id in the body, including someone else's. Closing it properly means giving Student a foreign key to User and overriding perform_create to ignore the body's value. See [chapter 45](#).

16.4 Where the classes are attached

Permission class	Views
IsAdmin	RegisterView
IsAuthenticated	ProtectedView , ChangePasswordView
AllowAny	LoginView , PasswordResetRequestView , PasswordResetConfirmView
AdminWrites	Teacher ×2, Student ×2
TeachingStaffWrites	Course ×2, Enrollment ×2, Lesson ×2, Assignment ×2, Results ×2
SubmissionWrites	Submission ×2

"×2" is the list view and the detail view. Both get the same class — forgetting the detail route is the classic way to leave a hole, and the smoke test checks every detail route separately for exactly that reason.

16.5 The mirror in the browser

`frontend/src/permissions.js` encodes the same table in JavaScript and is used only to decide which buttons to draw. A mistake there is a cosmetic bug, not a hole. Change one file, change the other.

LMS

- Dashboard
- Teachers
- Students
- Courses**
- Enrollments
- Lessons
- Assignments
- Submissions
- Results

Courses
Every course, and who teaches it.

24 courses

Search: Teacher: Any teacher

ID ↑	TITLE	DESCRIPTION	TEACHER	ACTION
27	Full-Stack Web Development with Django	Build and ship a complete Django application, fr...	Demo Teacher	—
28	Modern JavaScript and React	Go from ES6 fundamentals to a React front end...	Kamrul Hasan	—
29	Data Structures and Algorithms in Python	The interview syllabus, taught as something yo...	Sabbir Ahmed	—
30	SQL and Relational Database Design	Design a schema that will not embarrass you in ...	Rezaul Karim	—
31	Data Analysis with Excel and Power BI	For people who need answers out of a spreads...	Tasnim Ara	—
32	Machine Learning Foundations	What the common models do, when they fail, a...	Imran Chowdhury	—
33	UI/UX Design with Figma	Research, wireframe, prototype and hand off. P...	Maliha Chowdhury	—
34	Graphic Design with Adobe Illustrator	Vector work from the pen tool to a print-ready b...	Anika Tabassum	—
35	Digital Marketing Fundamentals	Run a campaign end to end: audience, message...	Farhan Islam	—
36	Spoken English for Professionals	Speak confidently in meetings, interviews and c...	Israt Ferdous	—

Showing 1–10 of 24 Rows 10 << < 1 2 3 > >>

demo_student Student Log out

The Students page signed in as a student: no Add button, no row actions, and no Accounts link in the sidebar. The API refuses those calls too.

17. `filters.py`

DRF ships a search filter and an ordering filter but not a field filter, and `django-filter` is a dependency this project does not have. This file is the missing piece, in about 130 lines.

17.1 How a view declares its filters

PYTHON

```
class AssignmentListCreateView(generics.ListCreateAPIView):
    filter_fields = {
        "course": "course",          # ?course=3
        "lesson": "lesson",          # ?lesson=7
        "due_from": "due_date__gte",  # ?due_from=2026-01-01
        "due_to": "due_date__lte",    # ?due_to=2026-03-31
    }
```

The key is the query parameter the caller types; the value is the ORM lookup it turns into. Anything not on the dictionary is ignored — there is no way to filter on a column the view did not name. Ranges are two parameters against one column, spelled with `__gte` and `__lte`.

17.2 `field_behind(model, lookup)`

Walks a lookup path and returns the model field at the end of it, so the value can be coerced to the right Python type.

PYTHON

```
parts = lookup.split("__")
for index, part in enumerate(parts):
    try:
        field = model._meta.get_field(part)
    except FieldDoesNotExist:
        break
    # the rest is a lookup (gte, icontains, in)
    if field.is_relation and index < len(parts) - 1:
        model = field.related_model
```

Lookup	Walk	Field found
<code>is_active</code>	one hop	<code>BooleanField</code>
<code>due_date__lte</code>	<code>due_date</code> , then <code>lte</code> is not a field → stop	<code>DateTimeField</code>
<code>submission__student</code>	hop the relation, then <code>student</code>	<code>ForeignKey</code>
<code>results__isnull</code>	<code>results</code> is the reverse relation, <code>isnull</code> is a lookup	the reverse relation

17.3 `coerce(field, lookup, raw, param)`

Query-string values arrive as strings. Passing them through would be a real bug rather than a cosmetic one: `?is_active=false` would filter on the non-empty string `"false"`, which is truthy, and so would match every row. The coercion rules, in the order they are tried:

Condition	Becomes
Lookup ends in <code>__isnull</code> , or the field is a <code>BooleanField</code>	True for <code>true/1/yes/on</code> , False for <code>false/0/no/off</code> , else 400
The field is a relation	<code>int</code> — a foreign key is filtered by the id on the other end
<code>IntegerField</code>	<code>int</code>
<code>FloatField</code> / <code>DecimalField</code>	<code>float</code>
<code>DateField</code> / <code>DateTimeField</code>	left as a string — Django parses ISO dates itself and complains if it cannot
anything else	the stripped string

The `__isnull` test is checked *before* the relation test, and on purpose: without it, `results__isnull` would be seen as a relation and `"true"` would be parsed as an id, raising `ValueError` .

A value that cannot be coerced becomes a 400 naming the offending parameter, rather than a 500 out of the database driver:

```
JSON
{"is_active": "'maybe' is not a valid value for this filter."}
```

17.4 `FieldFilterBackend.filter_queryset`

1. Read `view.filter_fields` . Not declared? Return the queryset untouched — which is why the detail routes and the auth views are unaffected by this backend running globally.
2. For each declared parameter, read it off the query string. **Missing and empty mean the same thing** — the dropdown is on "Any" — so both are skipped. Filtering on `""` would match nothing, which is the opposite of what the user meant.
3. Coerce each value, collect into one dict, apply as a single `queryset.filter(**conditions)` .
4. Wrap that call: an unparseable date lands as a Django `ValidationError` here rather than in `coerce()` , and is turned into a 400.

There is also a `get_schema_operation_parameters()` so the filters would show up in a generated OpenAPI schema. Nothing in the project generates one today.

18. `pagination.py`

One class, forty lines, and the reason every list response is an object rather than an array.

PYTHON

```
class StandardPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = "page_size"
    max_page_size = 200

    def get_paginated_response(self, data):
        return Response({
            "count": self.page.paginator.count,
            "page": self.page.number,
            "page_size": self.get_page_size(self.request),
            "total_pages": self.page.paginator.num_pages,
            "next": self.get_next_link(),
            "previous": self.get_previous_link(),
            "results": data,
        })
```

Key	Meaning
<code>count</code>	Rows matching after filtering , not rows on this page. The table footer and the dashboard tiles both read this.
<code>page</code> , <code>total_pages</code>	Not in DRF's default envelope. Without them the frontend would have to parse the <code>next</code> URL to work out where it is.
<code>next</code> , <code>previous</code>	Absolute URLs, or <code>null</code> . <code>listAll()</code> in <code>api.js</code> walks <code>next</code> until it is null.
<code>results</code>	The rows.

Ten rows fits a laptop screen without scrolling the page. `?page_size=` raises it, capped at 200 so that one request cannot ask for the whole table and undo the point of paginating it. That cap is duplicated in `api.js` as `BIGGEST_PAGE`, which is the page size `listAll()` uses when walking.

NOTE

Asking for a page past the end is a 404 with the message *"Invalid page"*. The frontend catches exactly that and steps back one page — see `stepBackAfter` in [chapter 29](#) — because it happens naturally when you delete the last row on the last page.

19. `views.py`

Six auth views and sixteen resource views. The resource views are almost entirely declarative: a queryset, a serializer, a permission class, and four attributes that tell the filter backends what to do.

19.1 `blacklist_user_tokens(user)`

PYTHON

```
tokens = OutstandingToken.objects.filter(user=user)
for token in tokens:
    BlacklistedToken.objects.get_or_create(token=token)
```

Called after a password change and after a password reset. It blacklists every *refresh* token the user has been issued. The whole body is inside a bare `try/except Exception: pass`, so a missing blacklist app cannot break a password change.

WARNING

This does **not** revoke the access token already in the caller's hands. JWTs are verified by signature, not looked up in a table, so an issued access token stays valid until it expires — up to eight hours here. Both places that call this therefore also make the frontend sign the user out. That is a real mitigation, not a complete one: a token already copied out of `localStorage` keeps working.

19.2 The auth views

View	Route	Permission	Behaviour
RegisterView	POST /register/	IsAdmin	CreateAPIView over RegisterSerializer. 401 anonymous, 403 teacher/student.
LoginView	POST /login/	AllowAny	Look up the Profile by phone, check the password, return tokens
ProtectedView	GET PATCH /profile/	IsAuthenticated	Your own account, both ways
ChangePassword-View	POST /change-password/	IsAuthenticated	Needs the old password; blacklists refresh tokens
PasswordResetRequestView	POST /password-reset/	AllowAny	Mails a link; always answers 200
PasswordResetConfirmView	POST /password-reset-confirm/	AllowAny	Validates uid+token, sets the password, blacklists

LoginView in full

```
PYTHON
try:
    profile = Profile.objects.get(phone=phone)
    user = profile.user
except Profile.DoesNotExist:
    return Response({'error': 'Invalid phone or password'}, status=400)

if not user.check_password(password):
    return Response({'error': 'Invalid phone or password'}, status=400)

tokens = get_tokens_for_user(user)
return Response({
    'message': 'Login successful',
    'user_id': user.id,
    'username': user.username,
    'role': role_of(user),
    'tokens': tokens,
})
```

Three things to notice. The error message is identical for an unknown phone and a wrong password, so the endpoint does not disclose which numbers are registered. The tokens are **nested under tokens** — it is `data.tokens.access`, not `data.access`, which is the single most common mistake when writing a client against this API. And `role` is included so the frontend can decide which buttons to draw; it is not what enforces anything.

ProtectedView

`GET` returns a flat user object including the phone from the Profile and the role from `role_of()`. `PATCH` runs `ProfileUpdateSerializer` and then re-reads the user from the database before answering. It only ever touches `request.user`, so there is no way to aim it at somebody else's account.

The password reset pair

The request view answers `200` with *"If an account exists with this email, a password reset link has been sent."* whether or not the address matches anything. Only if a user is found does it build a link and send mail:

```
PYTHON
uid = urlsafe_base64_encode(force_bytes(user.pk))
token = default_token_generator.make_token(user)
reset_link = f"{settings.FRONTEND_PASSWORD_RESET_URL}?uid={uid}&token={token}"
```

In development that mail is printed to the terminal running `runserver`. Note `fail_silently=False`: with a real SMTP backend misconfigured, this endpoint will raise rather than lie about having sent something.

19.3 The resource views

Eight pairs, all built from two DRF generics:

PYTHON

```

class TeacherListView(generics.ListCreateAPIView):          # GET list, POST create
class TeacherRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
                                                    # GET, PUT, PATCH, DELETE

```

The list view of each pair declares four attributes, and those four are the entire difference between one resource and the next:

Attribute	Read by	What it controls
<code>search_fields</code>	<code>SearchFilter</code>	What <code>?search=</code> looks through (case-insensitive contains)
<code>filter_fields</code>	<code>FieldFilterBackend</code>	Which <code>?param=</code> filters exist and what they map to
<code>ordering_fields</code>	<code>OrderingFilter</code>	The whitelist for <code>?ordering=</code> ; stops a caller sorting on a column that is not on the table
<code>ordering</code>	<code>OrderingFilter</code>	The default sort. Paging needs a stable order , or a row can appear on two pages or on neither.

Every view's declarations, in one table

Re-source	Queryset	<code>search_fields</code>	<code>filter_fields</code>
Teacher	<code>plain</code>	<code>name, email, subject</code>	<code>is_active, subject</code> (icontains)
Student	<code>plain</code>	<code>name, email, roll_number</code>	<code>is_active, enrolled_from, enrolled_to</code>
Course	<code>select_related("teacher")</code>	<code>title, description, teacher__name</code>	<code>teacher</code>
Lesson	<code>select_related("course")</code>	<code>title, description, course__title</code>	<code>course</code>
Assignment	<code>select_related("course", "lesson")</code>	<code>title, description, course__title, lesson__title</code>	<code>course, lesson, due_from, due_to</code>
Enrollment	<code>select_related("student", "course")</code>	<code>student__name, student__roll_number, course__title</code>	<code>student, course, enrolled_from, enrolled_to</code>
Submission	<code>select_related("assignment", "student")</code>	<code>content, student__name, assignment__title</code>	<code>assignment, student, course, submitted_from, submitted_to, ungraded</code>
Results	<code>select_related("submission", "submission__student", "submission__assignment")</code>	<code>feedback, submission__student__name, submission__assignment__title</code>	<code>submission, student, assignment, score_min, score_max</code>

All eight default to `ordering = ["id"]`.

The two clever filters

"course": "assignment__course" on submissions. A submission has no course of its own; it borrows the one its assignment belongs to. That one line is what makes the Submissions page's Course dropdown work.

"ungraded": "results__isnull" on submissions. Because `Results.submission` is one-to-one, a submission with nothing pointing back at it is one nobody has marked. `?ungraded=true` is the list the "new result" dropdown is built from. Asking the database that is far cheaper than sending 7,665 submissions and 4,630 results to the browser for it to work out the difference.

ROUGH EDGE

The eight `RetrieveUpdateDestroy` views use a plain queryset with no `select_related`. That is one extra query per relation on a single-row fetch — harmless in practice, and inconsistent with the list views above them.

20. `urls.py` — the route table

Twenty-two paths, all under `/api/`. Six for accounts, then eight pairs.

Path	Name	View
<code>login/</code>	login	<code>LoginView</code>
<code>register/</code>	register	<code>RegisterView</code>
<code>profile/</code>	protected	<code>ProtectedView</code>
<code>change-password/</code>	change-password	<code>ChangePasswordView</code>
<code>password-reset/</code>	—	<code>PasswordResetRequestView</code>
<code>password-reset-confirm/</code>	—	<code>PasswordResetConfirmView</code>
<code>teacher/</code> · <code>teacher/<int:pk>/</code>	teacher-list · teacher-detail	Teacher pair
<code>student/</code> · <code>student/<int:pk>/</code>	student-list · student-detail	Student pair
<code>course/</code> · <code>course/<int:pk>/</code>	course-list · course-detail	Course pair
<code>enrollment/</code> · <code>enrollment/<int:pk>/</code>	enrollment-list · enrollment-detail	Enrollment pair
<code>lesson/</code> · <code>lesson/<int:pk>/</code>	lesson-list · lesson-detail	Lesson pair
<code>assignment/</code> · <code>assignment/<int:pk>/</code>	assignment-list · assignment-detail	Assignment pair
<code>submission/</code> · <code>submission/<int:pk>/</code>	submission-list · submission-detail	Submission pair
<code>results/</code> · <code>results/<int:pk>/</code>	result-list · result-detail	Results pair

NOTE

Seven of the eight resources are singular in the URL; **results** is plural. `api.js` carries a comment about it for the same reason this book does: it is the one path you will get wrong from memory.

Every path has a trailing slash and Django's `APPEND_SLASH` is on by default, so a request to `/api/student` is redirected. A redirect on a `POST` loses the body, so always include the slash.

21. admin.py

All nine models are registered with the `@admin.register` decorator. Most just set `list_display` and a search or filter list. One does something load-bearing:

PYTHON

```
@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    list_display = ['user', 'phone', 'role']
    list_editable = ['role']
    list_filter = ['role']
    search_fields = ['user__username', 'phone']
```

`list_editable = ['role']` means role is a dropdown straight from the changelist, because promoting someone who has just registered is the one admin job this app really needs. **The Django admin is the only place a role can be changed.** The API refuses it from `/profile/`, and the React app has no screen for it.

22. migrations/

Migration	Generated on	What it does
0001_initial	Django 6.0.6	Creates Student , Teacher , Profile (without role)
0002_course_enrollment_...	Django 6.0.6	Creates the other six: Course, Enrollment, Lesson, Assignment, Submission, Results
0003_profile_role	Django 5.2.7	Adds Profile.role , default="student"
0004_exist-ing_accounts_become_admins	hand-writ-ten	A data migration: Profile.objects.update(role="admin")

22.1 The data migration is the interesting one

PYTHON

```
def make_existing_accounts_admins(apps, schema_editor):
    """Everyone who already had an account keeps full access."""
    Profile = apps.get_model("backend", "Profile")
    Profile.objects.update(role="admin")

def back_to_student(apps, schema_editor):
    Profile = apps.get_model("backend", "Profile")
    Profile.objects.update(role="student")
```

New accounts default to `student` , but the people who registered before roles existed could do everything, and locking them out on upgrade would be a surprise. So they are promoted once, here. It is written with a reverse function, so `migrate backend 0003` is not a dead end.

It uses `apps.get_model()` rather than importing `Profile` directly — the historical model, frozen as of this migration. Importing the live model would break the moment a later migration changed the table.

WARNING

On a database that already has real users, this migration makes **every existing account an admin**. That is correct for the upgrade it was written for and wrong for almost anything else. Check who exists before running it against data you did not seed.

23. The seeder

`management/commands/seed_data.py` — 488 lines — and `seed_catalog.py` — 470 lines of content it draws on. Between them they turn an empty database into something worth clicking through.

23.1 `seed_catalog.py` : the content, not the logic

Kept in a separate file so the command stays readable. Everything in it is invented, but shaped like a real Bangladeshi online-course provider's data so the UI looks plausible.

Constant	Size	What it is
<code>FIRST_NAMES</code>	58	Given names. Crossed with the surnames to build many distinct people from two short lists.
<code>LAST_NAMES</code>	29	Surnames. $58 \times 29 = 1,682$ possible people.
<code>COURSE_CATALOG</code>	16	Each entry: title, subject, description, and a list of <code>(lesson title, lesson description)</code> tuples. 102 lessons in total.
<code>ASSIGNMENT_BRIEFS</code>	~5	Generic briefs, cycled by lesson index.
<code>SUBMISSION_NOTES</code>	~6	Templates with a <code>{topic}</code> slot filled from the assignment title.
<code>FEEDBACK_BY_BAND</code>	4 bands	<code>high / good / pass / low</code> , several lines each, chosen by score.

Subjects are drawn from the catalogue and assigned round-robin to teachers, so every subject is covered before any is doubled up. That is why the teacher list does not read as random when you open it next to the course list.

23.2 The idempotency trick

PYTHON

```
def dice(*parts):
    """A stable 0-99 for a set of ids."""
    return crc32(":".join(str(p) for p in parts).encode()) % 100
```

This is the whole idea. Every "does this row exist yet / should this student have handed this in" decision calls `dice()` with the ids involved instead of drawing from a random stream. Same inputs, same answer, in this process and the next one.

Python's own `hash()` would not do: it is salted per process for strings, so it gives a different answer on every run. `crc32` is stable across processes and machines.

The consequence: a lesson that was skipped on the first run is skipped on every run, and a student who had handed in four of six assignments still has four of six afterwards. Rows are also found by a stable key before being created — a person by email, a course by title, a lesson by title within its course — so a second run adds nothing and duplicates nothing.

23.3 The phases, in order

#	Method	What it does
0	<code>_wipe()</code>	Only with <code>--fresh</code> . Deletes in child-first order, then removes non-superuser accounts on the two example.com domains.
1	<code>_seed_demo_accounts()</code>	The three demo logins. Repaired, not skipped: the password and phone are set on every run, and another account squatting on one of the numbers is moved off it.
2	<code>_seed_admins()</code>	Three named admin accounts.
3	<code>_seed_teachers(n)</code>	Teacher rows plus a matching login each. Tied together by email, because the schema has no link.
4	<code>_seed_students(n)</code>	Student rows, roll numbers <code>OSD-2026-0001</code> onward, continuing rather than restarting the series.
5	<code>_seed_courses()</code>	One course per catalogue entry, owned by a teacher whose subject matches.
6	<code>_seed_course_content()</code>	Lessons from the catalogue; roughly three in four get an assignment, with due dates walking forward a week per lesson.
7	<code>_seed_enrollments()</code>	1–4 courses per active student, 1–2 for an inactive one.
8	<code>_seed_submissions()</code>	Per-student "diligence" of 45–99 decides what fraction of their assignments they hand in.
9	<code>_seed_results()</code>	Grades most of what is older than five days, leaving a realistic marking backlog.

The whole `handle()` is wrapped in `@transaction.atomic`, so a failure halfway through leaves the database as it was.

23.4 The details worth copying

Working around `auto_now_add`

`Enrollment.enrollment_date` and `Submission.submitted_at` cannot be set on insert. So the seeder inserts with `bulk_create`, reads the new rows back, computes a date for each, and rewrites them with `bulk_update`:

PYTHON

```
Enrollment.objects.bulk_create(new_rows, batch_size=500)
fresh = list(Enrollment.objects.filter(id__in=[r.id for r in new_rows]))
for row in fresh:
    row.enrollment_date = (
        self.now - timedelta(days=5 + dice("when", row.student_id, row.course_id) * 7)
    ).date()
Enrollment.objects.bulk_update(fresh, ["enrollment_date"], batch_size=500)
```

Submissions get the same treatment, but placed relative to their own assignment's due date — usually a few days before it, sometimes just after:

PYTHON

```
drift = dice("drift", row.student_id, row.assignment_id) / 10.0 - 7.0
row.submitted_at = row.assignment.due_date + timedelta(days=drift)
```

Grades that never move

PYTHON

```
score = max(0.0, min(100.0, random.Random(submission.id).gauss(72, 15)))
```

A fresh `Random` seeded with the submission's own id — so a mark is a function of the row, not of when the seeder happened to reach it. A normal-ish distribution centred just above a pass. About 15% of eligible submissions are skipped (`dice("graded", ...) >= 85`) so the Results page has a backlog to show, and anything submitted in the last five days is left alone entirely.

Reading assignments back from the database

`_seed_submissions` queries `Assignment.objects.all()` rather than walking the catalogue. Courses seeded by an earlier run — or by an older version of the catalogue — have assignments too, and their students should be handing work in as well. This is why a database seeded over time can hold more courses than the catalogue currently lists: the seeder tops up and never removes without `--fresh`.

Phone numbers by prefix

Prefix	Who
0170...	Admins (and the three demo accounts, on fixed numbers)
0192...	Teacher accounts
0181...	Student accounts
0179...	Where a squatter gets moved to when a demo number is reclaimed

ROUGH EDGE

`DEMO_ACCOUNTS` exists twice: in `seed_data.py` with all three roles, and in `frontend/src/pages/Login.jsx` where the login page's one-click buttons come from. The comment says they are kept in sync by hand — and today they are not: the login page lists only the admin. The teacher and student logins still work, they are just not offered as buttons.

24. `scripts/api_smoke_test.py`

487 lines that drive every endpoint against a running server as all three roles and report what happened. The closest thing this project has to a test suite.

24.1 Running it

```
SHELL
python manage.py runserver 8001      # terminal one
python scripts/api_smoke_test.py    # terminal two
```

It needs `requests`, which is not in `requirements.txt` — install it separately if it is not already there. Exit codes: `0` all passed, `1` something failed, `2` could not reach the server or could not sign in (which usually means the database was never seeded).

24.2 How it is built

Piece	Role
Report	Counts passes, collects failures with the response body, prints a summary. <code>check(label, response, expected)</code> takes a status code or a tuple of acceptable ones.
Client	A signed-in caller. Logs in on construction, keeps the bearer token on a <code>requests.Session</code> , and asserts the role the server reports matches the one asked for. <code>Client()</code> with no role is the anonymous caller.
re-source_matrix(ids)	The data table the write checks are generated from: for each resource, a create payload, an update payload, the set of roles that may write, and optionally a separate set that may create.

24.3 What it checks, section by section

Section	Checks
Authentic- ation	All three demo logins work and return both tokens plus the right role; a wrong password, an unknown phone and a missing field are all 400; a garbage bearer token is 401.
Own pro- file	Each role reads its own profile and sees its own role; anonymous gets 401; a name edit round-trips; sending role does not change it ; a duplicate phone is 400; wrong / mismatched / weak passwords are all refused.
Password reset	A known and an unknown address give the <i>same</i> 200; a malformed address is 400; a forged token is 400.
Account creation	Anonymous 401, teacher 403, student 403; an admin gets 201 and the chosen role is applied; duplicate username and weak password are 400; the new account can then log in .
Reading	All eight lists: anonymous 401, then 200 for each of the three roles. Then one detail route per resource, plus a 404 for a missing id.
Writing	For each of the eight resources: anonymous cannot create; every role that should not create gets 403; the <i>least privileged role that is still allowed</i> does the create, so the permission class is exercised at its boundary; the row reads back, accepts PATCH and PUT; every role that should not edit gets 403 on PATCH and DELETE; and nonsense input is 400.
Cleanup	Deletes everything it made, children first, accepting 204 or 404.

24.4 The two ideas worth stealing

An expected 403 is a pass. The permission rules are part of what "working" means, so a create that succeeds when it should have been refused is a failure, not a bonus. Roughly half the checks in the file expect a refusal.

Placeholders resolved mid-run. A result needs a submission that exists, and the submission is created earlier in the same loop. The matrix writes `{"submission": "$submission"}` and a small `resolve()` swaps any `"$name"` for the id it stands for just before the request goes out.

NOTE

`backend/backend/tests.py` is still the empty `startapp` stub. There is no `manage.py test` suite; this script is the whole of the testing story. See [chapter 45](#).

PART FOUR

The frontend, file by file

A React app with no state library, no data-fetching library and no component library. Thirty-two files, and five of them explain the other twenty-seven.

25. Entry points and build config

25.1 `index.html`

One `<div id="root">` and one module script pointing at `/src/main.jsx`. Vite rewrites that during a build. The title is `LMS`; the favicon is a local SVG.

25.2 `src/main.jsx`

```
JSX
createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <AuthProvider>
      <App />
    </AuthProvider>
  </BrowserRouter>,
);
```

Three layers and the order matters. `BrowserRouter` is outermost because `AuthProvider`'s children use `Navigate` and `useLocation`. There is no `StrictMode` wrapper — worth knowing, because effects therefore run once in development rather than twice.

25.3 `src/index.css`

```
JAVASCRIPT
@import "tailwindcss";
```

The entire stylesheet. Tailwind 4 needs no config file and no `@tailwind base/components/utilities` triplet. Every style in the app is a utility class in JSX; there is not one custom CSS rule anywhere.

25.4 `vite.config.js`

```
JAVASCRIPT
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: { port: 5180 },
});
```

Two plugins, one setting. Port 5180 because 5173–5175 were taken by other projects. There is **no dev proxy** — the browser calls the Django origin directly, which is why CORS has to be configured on the server side.

25.5 `eslint.config.js`

Flat config on ESLint 10, extending the JS recommended set plus the react-hooks and react-refresh plugins. One custom rule:

```
JAVASCRIPT
```

```
'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z]' }],
```

ESLint's own `no-unused-vars` does not count a name being used inside JSX, so every imported component and icon would be reported as unused. `eslint-plugin-react` normally fixes that but does not support ESLint 10 yet, so capitalised names are ignored instead. The cost: a genuinely unused `import { Trophy }` will not be caught.

26. `api.js` — every request

One file, and the only place in the app that calls `fetch`. Read this and you know the entire surface the browser touches.

26.1 Session storage

Function	What it does
<code>getToken()</code>	Reads <code>lms_access_token</code> from <code>localStorage</code>
<code>saveSession(token, user)</code>	Writes the token and a JSON <code>{id, username, role}</code> under <code>lms_user</code>
<code>getSavedUser()</code>	Parses it back, returning <code>null</code> on anything unparseable
<code>clearSession()</code>	Removes both keys

`localStorage` rather than a cookie is what makes a page refresh keep you signed in — and also what makes the token readable by any script on the origin. See [chapter 44](#).

26.2 `readError(body, response)`

DRF reports failures in at least three shapes:

```
JSON
{"detail": "You do not have permission to perform this action."}
{"error": "Invalid phone or password"}
{"phone": ["This field is required."], "email": ["Enter a valid email address."]}
```

The first is DRF's own for permissions and 404s, the second is the hand-written login view, the third is any serializer rejecting a field. `readError` flattens all of them into one readable string — `field: message` per line, with `non_field_errors` printed bare — which is what every `<Alert>` in the app ends up showing. Nothing else in the frontend has to know DRF's error shapes.

26.3 `request(path, options)`

The single choke point. In order:

1. Set `Content-Type: application/json` if there is a body.
2. Attach `Authorization: Bearer <token>` unless the caller passed `auth: false` (login and the two reset endpoints).
3. `fetch`. A network-level failure is caught and re-thrown as *"Could not reach the server. Is Django running on http://127.0.0.1:8001?"* — the message you see when the backend is not up.
4. **401 handling:** clear the session and dispatch a `window` event called `lms:unauthorised`, then throw *"Your session has expired. Please log in again."*
5. 204 → return `null`. `DELETE` has no body to parse.
6. Parse the text as JSON, tolerating an empty or non-JSON body.

7. Not OK → throw `new Error(readError(...))`. Every page catches that and puts `problem.message` in an alert.

NOTE

Step 4 is where the missing refresh endpoint shows up. There is nothing to retry with, so the only honest response to a rejected token is to drop the session. The event is how `AuthContext` hears about it without `api.js` having to import React.

26.4 The named auth calls

Function	Request
<code>login(phone, password)</code>	<code>POST /login/</code> , <code>auth: false</code>
<code>register(details)</code>	<code>POST /register/</code> — carries the admin's own token
<code>fetchProfile()</code>	<code>GET /profile/</code>
<code>updateProfile(values)</code>	<code>PATCH /profile/</code>
<code>changePassword(values)</code>	<code>POST /change-password/</code>
<code>requestPasswordReset(email)</code>	<code>POST /password-reset/</code> , <code>auth: false</code>
<code>confirmPasswordReset(details)</code>	<code>POST /password-reset-confirm/</code> , <code>auth: false</code>

26.5 `toQueryString(params)`

JAVASCRIPT

```
// {page: 2, search: "", course: 3} → "?page=2&course=3"
```

Empty values are dropped rather than sent as blanks. An empty `?search=` or `?course=` is the filter sitting on "Any", and the server ignores those anyway, so leaving them out keeps the address bar and the network tab readable.

26.6 `resource(path)` — five calls built once

JAVASCRIPT

```
function resource(path) {
  return {
    list: (params) => request(`/${path}/${toQueryString(params)}`),
    listAll: (params) => everyPage(path, params),
    count: async (params) => { /* page_size=1, read .count */ },
    create: (values) => request(`/${path}/`, {method:"POST", ...}),
    update: (id, values) => request(`/${path}/${id}/`, {method:"PUT", ...}),
    remove: (id) => request(`/${path}/${id}/`, {method:"DELETE"}),
  };
}
```

Eight resources are then one line each:

JAVASCRIPT

```

export const teachersApi    = resource("teacher");
export const studentsApi    = resource("student");
export const coursesApi     = resource("course");
export const enrollmentsApi = resource("enrollment");
export const lessonsApi     = resource("lesson");
export const assignmentsApi = resource("assignment");
export const submissionsApi = resource("submission");
export const resultsApi     = resource("results"); // note: plural

```

The three read shapes, and when to use which

Call	Returns	Costs	Used by
<code>list(params)</code>	The whole envelope, untouched	One request, one page	Every table
<code>listAll(params)</code>	Just the rows, every page walked	$\lceil n/200 \rceil$ requests	Dropdowns and id→name lookups
<code>count(params)</code>	A number	One request, one row	The dashboard's eight tiles

`update` is a `PUT`, not a `PATCH` — every form sends the complete row, so there is no partial-update path in the frontend at all.

`everyPage()`

JAVASCRIPT

```

for (let page = 1; ; page += 1) {
  const body = await request(`${path}/${toQueryString({...params, page, page_size: 200})}`);
  if (Array.isArray(body)) return body; // an unpaginated endpoint
  rows.push(...(body?.results ?? []));
  if (!body?.next) return rows;
}

```

200 is `BIGGEST_PAGE`, kept in step with `max_page_size` in `pagination.py`. The `Array.isArray` branch is defensive: an endpoint that was never paginated would answer with a bare array.

NOTE

`listAll` is deliberate, not an oversight. The tables page through their data, but the dropdowns need the whole list — a course row carries `{"teacher": 3}`, and a lookup list that is itself paginated cannot answer for a row on page 40. Those lists are small (26 teachers, 24 courses, 180 students), and this is the only place that pays for them.

27. `auth.js` and `AuthContext.jsx`

27.1 Why they are two files

`auth.js` holds the context object and the `useAuth()` hook. `AuthContext.jsx` holds the provider component. They are split because **Vite's fast refresh gives up on any file that mixes components with other exports** — keeping the `.jsx` file to a single component export keeps hot reload working.

JAVASCRIPT

```
export const AuthContext = createContext(null);

export function useAuth() {
  const value = useContext(AuthContext);
  if (!value) throw new Error("useAuth must be used inside an AuthProvider");
  return value;
}
```

The throw turns a missing provider into a clear error at the point of use instead of a confusing `null` dereference three components later.

27.2 What the provider holds

JAVASCRIPT

```
const [user, setUser] = useState(() => getSavedUser());
const [isLoggedIn, setIsLoggedIn] = useState(() => Boolean(getToken()));
```

Both initialised lazily from `localStorage`, which is what makes a page refresh not sign you out. The context value is `{user, isLoggedIn, login, logout}` and `user` is only ever `{id, username, role}` — the full account lives on the server and is fetched by the Profile page.

27.3 The two effects

Listening for a dead token

JAVASCRIPT

```
window.addEventListener("lms:unauthorised", handleUnauthorised);
```

That is the event `api.js` fires on any 401. The handler clears `user` and `isLoggedIn`, which makes `ProtectedRoute` redirect to `/login` on the next render. No import cycle between `api.js` and React, because the browser's event bus sits between them.

Topping up a role-less session

A session saved before roles existed has no `role` on it. Rather than treat that account as having no permissions, the provider calls `fetchProfile()` once, writes the answer back to `localStorage`, and sets state. It is guarded by a `stillMounted` flag so a late answer cannot set state after unmount.

27.4 `logIn` and `logOut`

JAVASCRIPT

```
async function logIn(phone, password) {  
  const data = await loginRequest(phone, password);  
  const nextUser = { id: data.user_id, username: data.username, role: data.role };  
  saveSession(data.tokens.access, nextUser); // nested. data.access does not exist.  
  setUser(nextUser);  
  setIsLoggedIn(true);  
}
```

`logOut` just calls `clearSession()` and resets both pieces of state. **There is no logout endpoint.** Discarding the token is all the frontend can do; the token stays valid on the server until it expires. The refresh token from the login response is never stored at all.

28. permissions.js

A deliberate copy of `backend/permissions.py`, used only to decide which buttons are worth drawing.

JAVASCRIPT

```
const WRITERS = {
  teacher: [ADMIN],
  student: [ADMIN],
  course: [ADMIN, TEACHER],
  enrollment: [ADMIN, TEACHER],
  lesson: [ADMIN, TEACHER],
  assignment: [ADMIN, TEACHER],
  submission: [ADMIN, TEACHER],
  results: [ADMIN, TEACHER],
};

export function canWrite(role, resource) {
  return (WRITERS[resource] ?? []).includes(role);
}

export function canCreate(role, resource) {
  if (resource === "submission" && role === STUDENT) return true;
  return canWrite(role, resource);
}
```

`canCreate` differs from `canWrite` in exactly one place, mirroring `SubmissionWrites` on the server. Every resource page reads both:

JAVASCRIPT

```
const mayCreate = canCreate(user?.role, "teacher"); // the Add button
const mayEdit = canWrite(user?.role, "teacher"); // the row pencil and bin
```

`roleLabel(role)` turns the stored value into "Admin" / "Teacher" / "Student", falling back to "No role" — that is the pill next to your name in the sidebar.

WARNING

A mistake in this file is a cosmetic bug, not a hole: the server checks again. But the two files must move together, or the app will offer buttons that fail with a 403.

29. useTableQuery.js

Everything the eight table pages need to ask the server for one page of rows, written once instead of eight times over.

29.1 What it holds

State	Meaning
page , pageSize	Which page and how big. PAGE_SIZES = [10, 25, 50, 100] .
searchBox	What the input shows. Updates on every keystroke.
search	What gets sent. Lags searchBox by 300 ms.
ordering	The sort, spelled as the server wants it: "name" or "-name" .
filters	An object of dropdown and date values.

29.2 The debounce

JAVASCRIPT

```
useEffect(() => {
  if (searchBox === search) return;
  const timer = setTimeout(() => {
    setSearch(searchBox);
    setPage(1);
  }, 300);
  return () => clearTimeout(timer);
}, [searchBox, search]);
```

Typing "sarah" is six keystrokes. Waiting a moment after the last one turns six requests into one. Two separate state values, not one, is what makes that possible: the input stays responsive while the request lags behind.

29.3 Everything resets the page

setPageSize , setFilter , toggleSort and the debounce all call setPage(1) . A new search is a new result set, so page 4 of the old one is meaningless.

29.4 toggleSort

JAVASCRIPT

```
setOrdering((current) => (current === field ? `-${field}` : field));
```

Click a header to sort by it; click the same one again to turn it around. The leading - is what OrderingFilter reads as descending. There is no third "unsorted" state.

29.5 `clear()` and `isFiltered`

`clear()` resets the search, the filters, the ordering and the page — but deliberately *not* the page size, which is a preference about the screen rather than part of the question being asked of the data. `isFiltered` is true when `clear()` would have something to undo, and is what makes the Clear button appear.

29.6 `params`, and why it is memoised

JAVASCRIPT

```
const params = useMemo(
  () => ({ page, page_size: pageSize, search, ordering, ...filters }),
  [page, pageSize, search, ordering, filters],
);
```

Every page lists `query.params` as a dependency of the effect that loads rows. An object rebuilt on every render would be a new reference every time and the effect would loop forever. This is the single most important line in the file.

29.7 `stepBackAfter(problem)`

JAVASCRIPT

```
if (page > 1 && /invalid page/i.test(problem?.message ?? "")) {
  setPage((current) => current - 1);
  return true;
}
return false;
```

Delete the only row on the last page and the browser is left asking for a page that no longer exists, which the server answers with a 404 saying "Invalid page". Every page's `catch` block calls this first and bails out if it returns true, so the reader sees the previous page rather than an error.

30. App.jsx — the routes

JSX

```

<Routes>
  <Route path="/login"          element={<Login />} />
  <Route path="/forgot-password" element={<ForgotPassword />} />
  <Route path="/reset-password" element={<ResetPassword />} />
  <Route path="/register"       element={<Navigate to="/login" replace />} />

  <Route element={<ProtectedRoute><Sidebar /></ProtectedRoute>}>
    <Route path="/"          element={<Dashboard />} />
    <Route path="/teachers"  element={<Teachers />} />
    ...
    <Route path="/profile"   element={<Profile />} />
    <Route path="/accounts" element={
      <ProtectedRoute role="admin"><Accounts /></ProtectedRoute>
    } />
  </Route>

  <Route path="*" element={<NotFound />} />
</Routes>

```

Path	Page	Guard
/login	Login	public — someone who has forgotten their password cannot be signed in
/forgot-password	ForgotPassword	
/reset-password	ResetPassword	
/register	—	redirects to /login ; it used to be a public sign-up page
/	Dashboard	inside the ProtectedRoute + Sidebar layout route: a token is required
/teachers	Teachers	
/students	Students	
/courses	Courses	
/enrollments	Enrollments	
/lessons	Lessons	
/assignments	Assignments	
/submissions	Submissions	
/results	Results	
/profile	Profile	
/accounts	Accounts	a second, nested ProtectedRoute role="admin"
*	NotFound	outside the layout, so there is no sidebar to escape with

30.1 The layout route

The `<Route element={...}>` with no `path` is a *layout route*. Its element renders once and the matched child appears wherever that element puts `<Outlet />` — which `Sidebar.jsx` does, in the main column. So one guard and one frame cover eleven pages.

30.2 ProtectedRoute

JSX

```
if (!isLoggedIn) {
  return <Navigate to="/login" replace state={{ from: location.pathname }} />;
}
if (role && !user?.role) {
  return null; // the role is still being topped up from /profile/
}
if (role && user.role !== role) {
  return <Navigate to="/" replace />;
}
return children;
```

Three decisions. `state.from` remembers where you were headed so the login page can send you back there instead of always dropping you on the dashboard. The middle branch matters because the role arrives a moment after the token on a restored session — without the wait, an admin refreshing on `/accounts` would be bounced to the dashboard by mistake. And the last one is why hiding the sidebar link is not enough: somebody can always type the URL.

31. Sidebar.jsx

The menu, the account block, and the frame every signed-in page renders inside.

31.1 The menu

A ten-item array of `{address, text, icon}`. One entry carries `onlyFor: ADMIN`:

```
JSX
{ address: "/accounts", text: "Accounts", icon: <UserPlus size={16} />, onlyFor: ADMIN }

const visibleMenu = menu.filter((item) => !item.onlyFor || item.onlyFor === user?.role);
```

The route and the API both check the role again. This is only about not offering it.

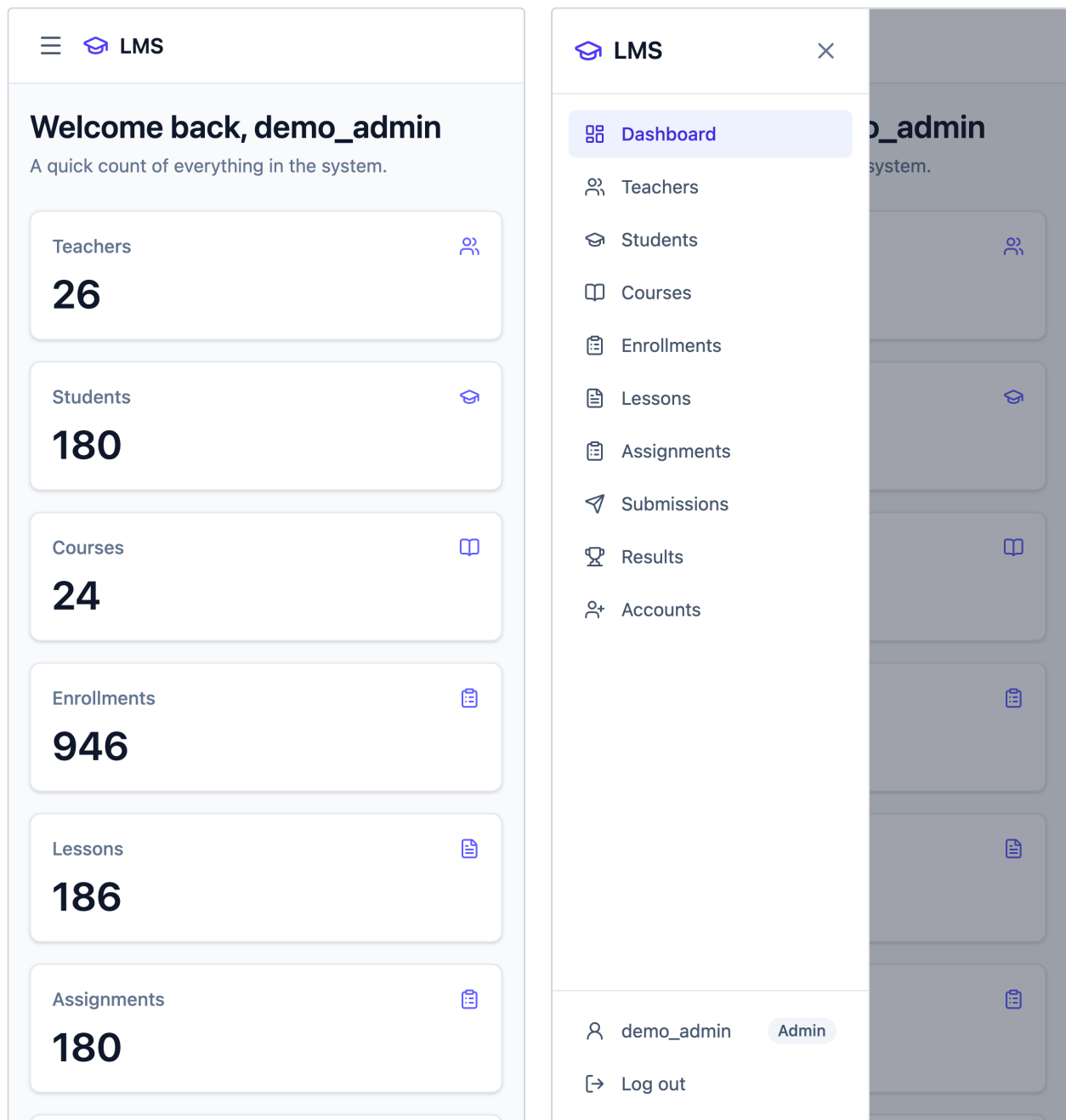
31.2 Two independent open/closed states

State	Applies to	Remembered?
<code>isCollapsed</code>	Wide screens (<code>md:</code> and up). Icon-only rail, 4rem wide.	Yes — <code>localStorage["lms:sidebarCollapsed"]</code>
<code>isDrawerOpen</code>	Narrow screens. Slides in over the page with a back-drop.	No — it is a moment, not a preference

Keeping them apart is why the collapsed classes carry `md:` prefixes rather than replacing the expanded ones: the drawer is always full width, so the icon-only layout is a wide-screen thing only.

31.3 The details

- **Escape closes the drawer**, matching the confirm dialog. The listener is only attached while it is open.
- **Tapping a link closes the drawer**, done in the link's own `onClick` rather than in an effect on the address — one event, not a rule about navigation.
- **Collapsed, the expand button takes the logo's place**. At 4rem there is room for one thing, and the way back out matters more than the branding.
- **The account block is a link to `/profile`**, showing the username and a role pill from `roleLabel()`. Log out sits below it.
- **The mobile top bar** — hamburger, logo, title — is `md:hidden` and is the only way to reach the menu on a phone.



The same app at 390 px: the sidebar becomes a drawer with a backdrop, and the tables scroll sideways.

32. components/ — the thirteen

All exported from `components/index.js`, so pages import one line: `import { Alert, Button, Table } from "../components/index.js";`

32.1 The shared bits

`styles.js` is four lines — the `inputBox` and `labelText` class strings, shared so every form control looks identical.

`field.js` holds `nameFromLabel()`, which turns "Confirm new password" into "confirm_new_password". A control wrapped in its own `<label>` already has an accessible name, so this is not about screen readers. It is about the two things a wrapper cannot give you: a `name`, which is what browser autofill and password managers key off, and an `id`, which is what the browser console asks for.

TIP

This is also how the screenshot script fills the login form: `page.fill('input[name="phone_number"]', ...)` works only because the label reads "Phone number".

32.2 The catalogue

Component	Lines	What it is
Button	22	Three looks: <code>primary</code> (indigo), <code>secondary</code> (outlined), <code>danger</code> (red). Defaults to <code>type="button"</code> so it cannot submit a form by accident.
IconButton	18	The same idea for a bare icon: <code>plain</code> and <code>danger</code> . Used for the row pencil and bin.
Input	66	Labelled text box. A <code>type="password"</code> automatically grows an eye toggle, so no page has to implement one.
Select	31	Labelled <code><select></code> with an optional placeholder option that has an empty value — which is what "Any" means to <code>toQueryString</code> .
Textarea	23	Labelled textarea, three rows by default.
Checkbox	22	Label wrapping the box; used for every <code>is_active</code> .
Alert	53	Three variants (error, success, info). Renders nothing when <code>children</code> is empty, so pages can leave it mounted with a state value in it. Pass <code>onDismiss</code> to get a close button.
PageHeader	8	An <code>h1</code> and a grey subtitle. Every screen starts with one.
ConfirmDialog	80	Replaces <code>window.confirm()</code> . Escape cancels, backdrop click cancels, the card stops propagation, and the confirm button shows "Deleting..." while it works.
FilterBar	60	The strip above a table: search box, the page's own dropdowns as <code>children</code> , and a Clear button that only appears when there is something to clear.
Table	104	Builds the header row from a column list and renders the body rows a page supplies.
Pagination	134	The strip below a table: "Showing 21–30 of 214", the rows-per-page select, and the pager.
Protected-Route	27	Covered in chapter 30 .

32.3 Input's password branch

The non-password path nests the input inside the label. The password path cannot: the eye button has to sit outside the `<label>`, or its own text becomes part of the box's accessible name and a screen reader reads out "Password Show password". So the password version ties label and input together with an `id` instead of by nesting, and the button's `aria-label` says what it will do next, not what the box is doing now.

32.4 Table's column format

```
JSX
columns=[["ID", { label: "Name", field: "name" }, "Action"]]
```

A plain string is a column there is no point sorting on. An object means sortable, and `field` is the name the server will accept in `?ordering=`. The header renders a button, sets `aria-sort` on the sorted column, and shows one of three icons: up, down, or a faint two-way chevron on the columns that *could* be sorted. Leave `onSort` out and every header is plain text.

`isEmpty` plus `emptyMessage` renders one centred row spanning every column. The pages pass a different message depending on whether a filter is active — "No teacher matches those filters" versus "No teachers yet".

32.5 Pagination's window

JAVASCRIPT

```
const WINDOW = 5;
let first = page - Math.floor(WINDOW / 2);
first = Math.max(1, Math.min(first, totalPages - WINDOW + 1));
```

With 22 pages and the reader on 11, the row reads `9 10 [11] 12 13` rather than every number from 1 to 22. The clamping keeps the window full at both ends: on page 1 it runs 1–5, on the last page it runs (last–4)–last, instead of trailing off. First / previous / next / last buttons flank it, disabled at the edges, each with an `aria-label`. The current page carries `aria-current="page"`.

33. The eight resource pages

Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results. They are the same page eight times. Read one and you have read them all.

33.1 The six steps

Both `Students.jsx` and `Teachers.jsx` carry this comment at the top, and all eight follow it:

1. **State** for the rows, the form, and the errors
2. `useEffect()` reads one page of rows from the API
3. `reload()` re-runs that effect after a save or a delete
4. `handleSave()` creates a new row, or updates the one being edited
5. `askToDelete()` / `confirmDelete()` opens the dialog, then deletes
6. **The JSX**: banners, form, filters, table, pages, confirm dialog

33.2 Step 1 — the state, every page

JSX

```
const [teachers, setTeachers] = useState([]); // this page of rows
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState("");
const [notice, setNotice] = useState(""); // the green "Saved" line

const [count, setCount] = useState(0); // rows matching the filters
const [totalPages, setTotalPages] = useState(1);

const query = useTableQuery({ ordering: "id" });
const { params: queryParams, stepBackAfter } = query;

const { user } = useAuth();
const mayCreate = canCreate(user?.role, "teacher");
const mayEdit = canWrite(user?.role, "teacher");

const [rowToDelete, setRowToDelete] = useState(null); // null = dialog closed
const [formIsOpen, setFormIsOpen] = useState(false);
const [editingId, setEditingId] = useState(0); // 0 = creating
const [reloadCount, setReloadCount] = useState(0);
```

`count` comes off the response, never from `teachers.length`, which only counts the page on screen. `editingId === 0` is the sentinel for "this form is a create, not an edit".

33.3 Step 2 — the load effect, and the race it avoids

```
JSX
useEffect(() => {
  let isCurrent = true;

  async function load() {
    setIsLoading(true);
    try {
      const body = await teachersApi.list(queryParams);
      if (!isCurrent) return;
      setTeachers(body.results);
      setCount(body.count);
      setTotalPages(body.total_pages);
      setError("");
    } catch (problem) {
      if (!isCurrent) return;
      if (stepBackAfter(problem)) return;
      setError(problem.message);
    } finally {
      if (isCurrent) setIsLoading(false);
    }
  }

  load();
  return () => { isCurrent = false; };
}, [reloadCount, queryParams, stepBackAfter]);
```

Typing in the search box fires a request per pause, and a slow one can land after a later, quicker one. The `isCurrent` flag, flipped by the cleanup function, makes the page ignore the answer to a question it has stopped asking. Without it the table can settle on the results for "sara" after you have finished typing "sarah".

33.4 Steps 4 and 5 — writing

```
JSX
const values = { name, email, subject, is_active: isActive };
if (editingId === 0) {
  await teachersApi.create(values); // POST
  setNotice("Teacher added.");
} else {
  await teachersApi.update(editingId, values); // PUT, the whole row
  setNotice("Teacher updated.");
}
setError(""); closeForm(); reload();
```

Delete is two functions: `askToDelete(row)` only sets the row into state, which opens the dialog; `confirmDelete()` does the request. On failure it closes the dialog and shows the error rather than leaving a modal stuck open.

33.5 What differs between the eight

Page	Lines	Lookups it loads	Filter controls
Teachers	367	none	Status
Students	421	none	Status, Enrolled from / to
Courses	371	teachers	Teacher
Lessons	372	courses	Course
Enrollments	414	students, courses	Student, Course, Enrolled from / to
Assignments	504	courses, lessons	Course, Lesson, Due from / to
Submissions	493	assignments, students, courses	Course, Assignment, Student, Handed in from / to
Results	490	ungraded submissions, students, assignments	Student, Assignment, Score from / to

33.6 The lookup effect

Six of the eight load lookup lists for their dropdowns, in a **second effect keyed only on `reloadCount`** — so turning a page does not re-read them:

```
JSX
useEffect(() => {
  async function loadLookups() {
    try {
      const [studentRows, courseRows] = await Promise.all([
        studentsApi.listAll({ ordering: "name" }),
        coursesApi.listAll({ ordering: "title" }),
      ]);
      setStudents(studentRows);
      setCourses(courseRows);
    } catch (problem) { setError(problem.message); }
  }
  loadLookups();
}, [reloadCount]);
```

These lists are for the *dropdowns only*. The name shown in each table row comes off the row itself as `teacher_name` / `course_title` / `student_name`, so no join is done in the browser.

33.7 The three pages that do something extra

Assignments — cascading dropdowns

```
JSX
const lessonsToOffer = courseId
  ? lessons.filter((lesson) => lesson.course === Number(courseId))
  : lessons;
```

An assignment points at both a course and a lesson, so once a course is chosen only that course's lessons are worth offering. The filter bar gets the same narrowing against its own course dropdown. The page also converts between the API's UTC timestamps and what `<input type="datetime-local">` wants:

JAVASCRIPT

```
function toDateTimeInputValue(text) {
  const moment = new Date(text);
  const offsetInMs = moment.getTimezoneOffset() * 60 * 1000;
  return new Date(moment.getTime() - offsetInMs).toISOString().slice(0, 16);
}
```

Slicing the ISO string directly would show the UTC clock time, so the offset comes off first.

Submissions — a course filter for a row that has no course

Picking a course narrows the assignment dropdown next to it, the same trick as above. The request it produces is `?course=3`, which the server maps to `assignment__course` — the submission borrows its assignment's course.

Results — grading the ungraded

JSX

```
const UNGRADED_TO_OFFER = 200;

submissionsApi.list({
  ungraded: "true",
  ordering: "submitted_at",
  page_size: UNGRADED_TO_OFFER,
})
```

Because `Results.submission` is one-to-one, grading the same submission twice is a 400. So the dropdown is built from `?ungraded=true` — the server answering "no result attached yet" from the database — and only the first 200 of them, oldest first. A `<select>` is no way to hunt through thousands; the count under the box says how many are waiting, and grading one brings the next into view.

One more wrinkle: the submission being *edited* is already graded, so it is not in the ungraded list. The page keeps `editingRow` — found in the results already on screen — so the dropdown can show its label instead of a blank.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

Results

Scores and feedback for submitted work.

4630 results

+ Add result

Search

Student

Assignment

Score from

Feedback, student or assignn

Any student

Any assignment

Score to

ID ↑	SUBMISSION ↕	SCORE ↕	FEEDBACK	ACTION
571	#677 — Demo Student, Case Study 4: Train/Test Splits and Cross-Validation	48	The approach is not quite right, though I can see the effort...	 
572	#678 — Demo Student, Project Milestone 5: Decision Trees and Ensembles	56	This only covers part of the brief. Please see me in the nex...	 
573	#679 — Demo Student, Lab 6: Evaluating a Model Honestly	87	Solid throughout. One small slip near the end, which I have ...	 
574	#680 — Demo Student, Quiz 7: Packaging a Model Behind an API	83	Good submission. The work is correct; tighten the write-up a...	 
575	#681 — Demo Student, Exercise 7: Packaging a Model Behind an API	65	The core is right, but parts of it are rushed. Re-read the l...	 
576	#682 — Demo Student, Case Study 4: State Management with Provider	79	Good submission. The work is correct; tighten the write-up a...	 
577	#683 — Demo Student, Project Milestone 5: Consuming REST APIs and Local Storage	99.5	One of the strongest submissions in the batch. Your reasonin...	 
578	#685 — Demo Student, Project Milestone 2: Keyword Research and On-Page SEO	82	Well done. Your explanation is clear — next time show the in...	 
579	#686 — Demo Student, Exercise 5: Email Marketing and Automation	68	Reasonable attempt. Your answer is correct in outline but th...	 

Results: the score and feedback attached to a submission, with the student and assignment names carried on the row by `ResultSerializer`.

34. The seven other screens

34.1 Dashboard.jsx

Eight tiles, one per resource, each a `<Link>` to its table. The counts come from one `Promise.all` over `tile.api.count()`:

```
JSX
const totals = await Promise.all(tiles.map((tile) => tile.api.count()));
```

There is still no counts endpoint on the server, but a paginated list reports how many rows matched, so `count()` asks for `page_size=1` and reads `count` off the envelope. Eight tiles used to mean downloading eight whole tables — 7,665 submissions among them. Now it is eight requests for one row each.

While loading, each tile shows an em dash rather than a zero, so "still counting" and "none" do not look the same.

34.2 Login.jsx

Phone and password, submitted through `useAuth().login`. Three details:

- **Where it sends you afterwards:** `location.state?.from ?? "/"` — the path `ProtectedRoute` stashed when it bounced you.
- **The notice:** `/reset-password` navigates here with `state.notice`, which is read *into state* so dismissing it sticks.
- **The dev-only panel** is wrapped in `import.meta.env.DEV`. Vite replaces that with `false` in a production build, so the whole block and the `DEMO_ACCOUNTS` array are dropped from the bundle. Clicking a demo button fills the fields as well as signing in, so it is obvious what was used.

There is no sign-up link, only the line *"No account? Ask an admin to make you one."*

34.3 ForgotPassword.jsx

Step one of two: one email field. On success the page swaps to a confirmation that says *"If an account uses ..., a reset link is on its way"* — **"if", not "we sent"**, because the server answers identically either way and the page must not disclose more than the server does.

In development it adds a dashed amber note: the mail backend prints to the Django terminal instead of sending, so the link is in that window.

34.4 ResetPassword.jsx

Step two. It reads `?uid=` and `?token=` with `useSearchParams` and hands them straight back to the server — checking them in the browser would mean nothing. If either is missing, it says the link is incomplete rather than posting a request that is certain to fail.

On success it navigates to `/login` with a notice, because resetting the password has retired the old sessions anyway.

34.5 Accounts.jsx

Admin only, twice over: `ProtectedRoute role="admin"` in the router, and `IsAdmin` on `/api/register/`. Seven fields plus a role select whose options spell out what each role can do.

After a successful create it shows the phone number the new person will sign in with, and clears the form. It **deliberately does not log you in as the new account** — the admin who filled it in stays signed in as themselves.

The screenshot shows the LMS interface. On the left is a sidebar with a menu: Dashboard, Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results, and Accounts (highlighted). At the bottom of the sidebar, it shows 'demo_admin' with the role 'Admin' and a 'Log out' button. The main content area is titled 'Accounts' with the subtitle 'Create a login for a student, a teacher, or another admin.' Below this is a 'New account' form. The form has fields for 'First name', 'Last name', 'Username', 'Email', 'Phone' (with the value '01711110001'), and 'Password' (with an eye icon for toggling visibility). There is a 'Role' dropdown menu currently set to 'Student — read only, may hand work in'. Below the form, a note says 'They sign in with the phone number, not the username.' At the bottom of the form are two buttons: 'Create account' (with a plus icon) and 'Clear'.

Creating a login. The role picked here is the one the API applies, because `RegisterSerializer` accepts it and the view above it is admin-only.

34.6 Profile.jsx

Two separate forms side by side, because they are two different jobs with different rules.

Your details — first name, last name, email, phone — read from `GET /profile/` rather than from the saved session, which only holds id, username and role. Saving is a `PATCH`. Username and role are shown but not editable, with a line saying why.

Change password — current, new, confirm. On success the page calls `logout()` and navigates to `/login`. The old access token stays technically valid until it expires, so the honest thing is to end the session here rather than pretend it was revoked.

An admin sees one extra line at the bottom: changing somebody else's details, or anybody's role, is done from the Django admin site.

34.7 `NotFound.jsx`

Sits outside the layout route, so there is no sidebar to escape with — which is what the button is for. Where it points depends on whether there is a session: the dashboard needs one, the login page does not, and offering the wrong one only bounces you straight back.

PART FIVE

How it behaves

Four cross-cutting stories that no single file tells: the session, the list pipeline, the permission model, and what performance cost what.

35. The authentication lifecycle

35.1 Signing in

```

POST /api/login/ {"phone": "01700000000", "password": "admin1234"}
  |
  ├── Profile.objects.get(phone=...) → the User behind it
  ├── user.check_password(...) → 400 "Invalid phone or password" if wrong
  └── RefreshToken.for_user(user) → {access, refresh}

200 {"message": "Login successful", "user_id": 1, "username": "demo_admin",
    "role": "admin", "tokens": {"access": "eyJ...", "refresh": "eyJ..."}}
  |
  └── AuthContext.login():
        saveSession(data.tokens.access, {id, username, role})
          |
          └── localStorage: lms_access_token = "eyJ..."
                        lms_user = {"id": 1, "username": "demo_admin", "role": "admin"}

```

The refresh token is received and thrown away. Nothing in the frontend stores it, because there is nothing to exchange it at.

35.2 While signed in

Every request through `request()` attaches `Authorization: Bearer <access>`. The server verifies the signature and the expiry — it does *not* look the token up in a table, which is the whole point of a JWT and also the whole limitation.

35.3 Signing out, three ways

Trigger	What happens	Is the token really dead?
The Log out button	<code>clearSession()</code> , state reset, router redirects	No — it stays valid on the server until it expires
Any 401 from any request	<code>api.js</code> clears the session and fires <code>lms:unauthorised</code> ; the provider drops the user	Yes — that is what the 401 means
Changing your password	Server blacklists every refresh token; the page logs you out and navigates to <code>/login</code>	Partly — refresh tokens yes, the live access token no

WARNING

The honest summary: **an access token cannot be revoked in this design**. Up to eight hours after a password change, a token copied out of `localStorage` beforehand still works. Shortening `ACCESS_TOKEN_LIFETIME` and adding a refresh endpoint is the standard fix, and it is the single largest change this project would need before facing anything real.

35.4 The password reset round trip

```

/forgot-password —POST /api/password-reset/ {"email": "..."}→ always 200
                                |
                                | user found? uid = b64(pk), token = default_token_generator
                                |
                                |← email (console backend in dev)
                                |
                                | http://localhost:5180/reset-password?uid=MQ&token=cvx2p1-...
                                |
/reset-password —POST /api/password-reset-confirm/ {uid, token, new, confirm}→
                                |
                                | decode uid → user, check_token (1 hour window),
                                | validate_password, set_password, blacklist refresh tokens
                                |
                                |← 200 — navigate to /login with a notice

```

The one-hour window is `PASSWORD_RESET_TIMEOUT`. The token also becomes invalid as soon as the password changes, because Django's generator hashes the current password into it — so a link cannot be replayed.

36. The list pipeline

Four query parameters, four stages, one envelope. Every list endpoint in the project behaves identically.

```
GET /api/student/?search=nusrat&is_active=true&ordering=-name&page=2&page_size=25

queryset = Student.objects.all()
|
1 SearchFilter      ?search=      → OR of icontains over search_fields
  |                  (name, email, roll_number)
2 FieldFilterBackend ?is_active=  → declared in filter_fields, coerced to a bool
  |
3 OrderingFilter    ?ordering=    → checked against ordering_fields; "-" = descending
  |                  default ["id"] when absent
4 StandardPagination ?page=      → COUNT(*) for the envelope, then LIMIT/OFFSET
  |
{"count": 214, "page": 2, "page_size": 25, "total_pages": 9,
 "next": "...?page=3&...", "previous": "...?page=1&...", "results": [ ... 25 rows ... ]}
```

36.1 The rules that fall out of that order

- **count is post-filter.** It is what the table footer shows and what the dashboard tiles read.
- **Search and filter compose.** `?search=nusrat&is_active=true` is an AND, not a choice between them.
- **Ordering must be stable or paging breaks.** Every view sets `ordering = ["id"]` as its default; without one, a row can appear on two pages or on neither.
- **An undeclared parameter is silently ignored.** `?colour=blue` on the students list is not an error — it simply is not on the dictionary.
- **An empty parameter is the same as an absent one.** `?course=` means "Any", which is why the frontend can send the whole filter object without pruning it first.

36.2 Worked examples

SHELL

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8001/api/login/ \
-H 'Content-Type: application/json' \
-d '{"phone":"01700000000","password":"admin1234"}' \
| python3 -c 'import sys,json; print(json.load(sys.stdin)["tokens"]["access"])')

# Assignments due in the next fortnight, soonest first
curl -s -H "Authorization: Bearer $TOKEN" \
"http://127.0.0.1:8001/api/assignment/?due_from=2026-09-07&due_to=2026-09-21&ordering=due_date"

# The marking backlog, oldest first
curl -s -H "Authorization: Bearer $TOKEN" \
"http://127.0.0.1:8001/api/submission/?ungraded=true&ordering=submitted_at&page_size=50"

# Everyone who failed a particular assignment
curl -s -H "Authorization: Bearer $TOKEN" \
"http://127.0.0.1:8001/api/results/?assignment=12&score_max=49&ordering=score"

# How many active students there are, without downloading any of them
curl -s -H "Authorization: Bearer $TOKEN" \
"http://127.0.0.1:8001/api/student/?is_active=true&page_size=1" | head -c 60
```

LMS

Dashboard
Teachers
Students
Courses
Enrollments
Lessons
Assignments
Submissions
Results
Accounts

demo_admin Admin

Log out

Students

Everyone enrolled in the school.

9 students

+ Add student

Search
rahman

Status
Active

Enrolled from
mm/dd/yyyy

Enrolled to
mm/dd/yyyy

Clear

ID	NAME	EMAIL	ROLL NO.	ENROLLED	ACTIVE	ACTION
214	Adnan Rahman	adnan.rahman.s@student.example.com	OSD-2026-0144	2025-03-26	Yes	
149	Afsana Rahman	afsana.rahman.s@student.example.com	OSD-2026-0079	2024-11-21	Yes	
150	Badhon Rahman	badhon.rahman.s@student.example.com	OSD-2026-0080	2025-10-28	Yes	
74	Hasibul Rahman	hasibul.rahman@student.example.com	OSD-2026-0004	2026-07-31	Yes	
156	Ishrat Rahman	ishrat.rahman.s@student.example.com	OSD-2026-0086	2025-06-09	Yes	
107	Noshin Rahman	noshin.rahman@student.example.com	OSD-2026-0037	2026-06-12	Yes	
190	Rezaul Rahman	rezaul.rahman.s@student.example.com	OSD-2026-0120	2025-03-09	Yes	
94	Rohan Rahman	rohan.rahman@student.example.com	OSD-2026-0024	2026-08-09	Yes	
168	Sanjida Rahman	sanjida.rahman.s@student.example.com	OSD-2026-0098	2026-07-12	Yes	

Showing 1–9 of 9

Rows
10
1

The same pipeline from the browser: search, two date filters, a status dropdown and a sorted column, all of it done on the server.

37. The permission matrix

The complete picture, with the file that enforces each cell.

Action	Anon	Student	Teacher	Admin
POST /login/	✓	✓	✓	✓
POST /password-reset/ , ...-confirm/	✓	✓	✓	✓
GET PATCH /profile/	401	✓	✓	✓
POST /change-password/	401	✓	✓	✓
POST /register/	401	403	403	✓
GET any resource list or detail	401	✓	✓	✓
Write /teacher/ , /student/	401	403	403	✓
Write /course/ , /lesson/ , /assignment/	401	403	✓	✓
Write /enrollment/ , /results/	401	403	✓	✓
POST /submission/	401	✓	✓	✓
PUT PATCH DELETE /submission/<id>/	401	403	✓	✓
Change anyone's role	—	—	—	Django admin only

37.1 401 versus 403

They mean different things and the smoke test asserts on the difference. **401** is "I do not know who you are" — no token, or a token the server will not accept. **403** is "I know exactly who you are, and no" — a valid token whose role is not on the list. If a call that should be 403 comes back 401, the authentication class is the problem, not the permission class.

37.2 Changing the rules

Three files move together, and in this order:

1. `backend/backend/permissions.py` — the actual rule. Change `roles_that_may_write`, or attach a different class to the view.
2. `frontend/src/permissions.js` — the mirror, so the buttons match.
3. `backend/scripts/api_smoke_test.py` — the `writers` / `creators` sets in `resource_matrix()`, or the test will now fail on the new behaviour.

38. Performance, and what it cost

Three decisions, each with a number behind it.

38.1 `select_related` on every joining list view

`ResultSerializer` reaches `submission.student.name` and `submission.assignment.title`. Without `select_related`, a ten-row page would be 1 query for the results plus 10 for the submissions plus 20 for their students and assignments. With it, one query with joins. The six views that touch relations all declare it.

38.2 Counting without downloading

The dashboard used to fetch all eight tables to display eight numbers — with the current data that is more than 13,000 rows on every visit to the home page. Now:

JAVASCRIPT

```
count: async (params) => {
  const body = await request(`${path}/${toQueryString({ ...params, page_size: 1 })}`);
  return Array.isArray(body) ? body.length : (body?.count ?? 0);
}
```

Eight requests, one row each. The count comes from the pagination envelope, which the database was computing anyway.

38.3 Names on the row instead of joins in the browser

Before pagination, the browser downloaded every teacher, course and student and did the id→name join itself. Two things broke that. A page of ten rows should not cost eight thousand. And once the lookup lists were paginated too, **a lookup list cannot answer for a row on page 40** — the name simply would not be there. Sending `teacher_name` alongside `teacher` costs nothing, because the join is already in the queryset.

What is still fetched in full: the dropdown lists, via `listAll()`. Those are 26 teachers, 24 courses, 180 students and 180 assignments — small, and needed whole because a `<select>` cannot page.

38.4 `?ungraded=true`

The Results page needs the submissions nobody has marked. Computing that in the browser means downloading 7,665 submissions and 4,630 results and subtracting one set from the other. As a query it is `results__isnull=True`, and the page asks for the first 200 of them.

38.5 The debounce

300 ms after the last keystroke, not on every one. Typing a six-letter name is one request instead of six — and the `isCurrent` flag means that even if two are in flight, the older answer cannot overwrite the newer one.

Change	Before	After
Opening a table page	the whole table	10 rows + a count
Opening the dashboard	8 whole tables (~13,800 rows)	8 requests, 1 row each
Rendering one row's related name	a browser-side join over a full lookup list	a field on the row
Finding ungraded submissions	7,665 + 4,630 rows, subtracted in JS	<code>?ungraded=true</code> , 200 rows
Typing "sarah"	6 requests	1

PART SIX

Working on it

Three recipes that cover most changes anyone will want to make, then testing, troubleshooting, and an honest list of what is not finished.

39. Recipe — add a field

Worked example: give `Teacher` a `joined_on` date that shows in the table, is editable in the form, and can be sorted on.

Step 1 — the model

```
PYTHON
# backend/backend/models.py
class Teacher(models.Model):
    ...
    joined_on = models.DateField(null=True, blank=True)
```

`null=True` matters: there are already 26 rows, and a non-nullable date would force you to invent a default for all of them.

Step 2 — the migration

```
SHELL
python manage.py makemigrations backend
python manage.py migrate
```

Step 3 — the serializer

```
PYTHON
# backend/backend/serializers.py
class TeacherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Teacher
        fields = ['id', 'name', 'email', 'subject', 'is_active', 'joined_on']
```

The `fields` list is explicit everywhere in this project, so a new column is invisible to the API until it is named here.

Step 4 — the view, if it should be sortable or searchable

```
PYTHON
# backend/backend/views.py - TeacherListCreateView
ordering_fields = ["id", "name", "email", "subject", "is_active", "joined_on"]
```

Step 5 — the page

Four edits in `frontend/src/pages/Teachers.jsx` :

JSX

```
// 1. state
const [joinedOn, setJoinedOn] = useState("");

// 2. both form openers
function openEmptyForm() { ...; setJoinedOn(""); ... }
function openFormForEditing(teacher) { ...; setJoinedOn(teacher.joined_on ?? ""); ... }

// 3. the payload – PUT sends the whole row, so a missing key clears the column
const values = { name, email, subject, is_active: isActive, joined_on: joinedOn || null };

// 4. the form control, the column header, and the cell
<Input label="Joined on" type="date" value={joinedOn}
  onChange={(e) => setJoinedOn(e.target.value)} />

columns=[..., { label: "Joined", field: "joined_on" }, "Action"]
<td className="px-3 py-2 text-slate-700">{teacher.joined_on}</td>
```

WARNING

`update()` is a `PUT`, not a `PATCH`. Forgetting to include the new key in `values` will not leave the column alone — it will blank it.

Step 6 — the smoke test, if the field is required

Add it to the `create` payload for that resource in `resource_matrix()`, or every create check will start failing with a 400.

40. Recipe — add a filter

Worked example: filter courses by whether their teacher is active.

Step 1 — declare it on the view

```
PYTHON
# backend/backend/views.py – CourseListCreateView
filter_fields = {
    "teacher": "teacher",
    "teacher_active": "teacher__is_active",    # new
}
```

That is the entire backend change. `FieldFilterBackend` walks `teacher__is_active`, finds a `BooleanField`, and coerces `"true"` to `True`. Test it immediately:

```
SHELL
curl -s -H "Authorization: Bearer $TOKEN" \
  "http://127.0.0.1:8001/api/course/?teacher_active=false"
```

Step 2 — add the control

```
JSX
<Select
  label="Teacher status"
  className="min-w-40"
  value={query.filters.teacher_active ?? ""}
  onChange={(event) => query.setFilter("teacher_active", event.target.value)}
>
  <option value="">Any</option>
  <option value="true">Active</option>
  <option value="false">Inactive</option>
</Select>
```

Drop that inside the page's `<FilterBar>` as a child. Nothing else is needed: `setFilter` already resets the page to 1, `params` already picks it up, the effect already re-runs, and `toQueryString` already drops the empty `"Any"` value.

40.1 Range filters

Two parameters against one column, and two date inputs:

```
PYTHON
"created_from": "created_at__gte",
"created_to": "created_at__lte",
```

JSX

```
<Input label="Created from" type="date" className="min-w-40"  
  value={query.filters.created_from ?? ""}  
  onChange={(e) => query.setFilter("created_from", e.target.value)} />
```

41. Recipe — add a whole resource

Nine files, in order. Worked example: `Announcement`, belonging to a course, writable by teaching staff.

1. Model — `backend/backend/models.py`

PYTHON

```
class Announcement(models.Model):
    course      = models.ForeignKey(Course, on_delete=models.CASCADE)
    title       = models.CharField(max_length=100)
    body        = models.TextField()
    posted_at   = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

2. Migration

SHELL

```
python manage.py makemigrations backend && python manage.py migrate
```

3. Serializer — `serializers.py`

PYTHON

```
class AnnouncementSerializer(serializers.ModelSerializer):
    course_title = serializers.CharField(source='course.title', read_only=True)

    class Meta:
        model = Announcement
        fields = ['id', 'course', 'title', 'body', 'posted_at', 'course_title']
```

Send the name beside the id. That is the convention here and it is what keeps the browser from having to join.

4. Views — `views.py`

PYTHON

```
class AnnouncementListCreateView(generics.ListCreateAPIView):
    queryset = Announcement.objects.select_related("course")
    serializer_class = AnnouncementSerializer
    permission_classes = [TeachingStaffWrites]

    search_fields = ["title", "body", "course__title"]
    filter_fields = {"course": "course",
                    "posted_from": "posted_at__gte",
                    "posted_to": "posted_at__lte"}
    ordering_fields = ["id", "title", "posted_at", "course__title"]
    ordering = ["id"]

class AnnouncementRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Announcement.objects.all()
    serializer_class = AnnouncementSerializer
    permission_classes = [TeachingStaffWrites] # ← the same class. Do not forget this.
```

5. Routes — `urls.py`

PYTHON

```
path('announcement/', AnnouncementListCreateView.as_view(), name='announcement-list'),
path('announcement/<int:pk>/', AnnouncementRetrieveUpdateDestroyAPIView.as_view(),
     name='announcement-detail'),
```

6. Admin — `admin.py`

PYTHON

```
@admin.register(Announcement)
class AnnouncementAdmin(admin.ModelAdmin):
    list_display = ['id', 'title', 'course', 'posted_at']
    list_filter = ['course']
    search_fields = ['title', 'body']
```

7. API client — `frontend/src/api.js`

JAVASCRIPT

```
export const announcementsApi = resource("announcement");
```

8. Permissions mirror — `frontend/src/permissions.js`

JAVASCRIPT

```
const WRITERS = {
  ...
  announcement: [ADMIN, TEACHER],
};
```

9. The page, the route, the menu entry

Copy `pages/Lessons.jsx` — it is the closest shape: one lookup list, one dropdown filter, a title, a description and a foreign key. Rename the eight or so identifiers, change the columns, change the form fields. Then:

```
JSX

// App.jsx, inside the layout route
<Route path="/announcements" element={<Announcements />} />

// Sidebar.jsx, in the menu array
{ address: "/announcements", text: "Announcements", icon: <Megaphone size={16} /> },

// Dashboard.jsx, in tiles
{ address: "/announcements", text: "Announcements", api: announcementsApi,
  icon: <Megaphone size={16} className="text-indigo-500" /> },
```

10. And the tests

Add an entry to `resource_matrix()` in the smoke test with a create payload, an update payload and `"writers": {"admin", "teacher"}`, and add the path to the list in `check_reads()`.

TIP

Checklist of the things that are easy to forget: the permission class on the *detail* view; the `select_related`; the trailing slash in the route; the mirror in `permissions.js`; and the smoke test entry.

42. Testing, linting, screenshots

42.1 The smoke test

```
SHELL
cd backend
python manage.py runserver 8001      # terminal one
python scripts/api_smoke_test.py     # terminal two
LMS_API=http://127.0.0.1:8002/api python scripts/api_smoke_test.py  # somewhere else
```

Covered in full in [chapter 24](#). It needs the demo accounts, so run `seed_data` first.

42.2 Django's own test runner

```
SHELL
python manage.py test
```

Runs, and finds nothing: `tests.py` is the empty stub. If you add tests there, note that Django will try to create a test database, which means the MySQL user needs `CREATE` privileges on `test_lms_db`.

42.3 Frontend

```
SHELL
cd frontend
npm run lint      # ESLint 10, flat config
npm run build     # production bundle into dist/ (gitignored)
npm run preview   # serve that bundle to check it
```

There is no frontend test runner. `npm run build` is the closest thing to a check — it fails on a syntax error or a bad import.

TIP

Build once before believing anything about the production bundle. Two things behave differently there: the login page's demo panel disappears (`import.meta.env.DEV`), and so does the dev-only note on the forgot-password page.

42.4 Redrawing the screenshots

```
SHELL
# both servers up, database seeded
pip install playwright && playwright install chromium
python3 docs/capture_screenshots.py
```

It drives a real Chromium at 1440×900 (and 390×844 for the two phone shots) at `device_scale_factor=2`, logs in as the admin and then as the student, visits every screen, and writes 18 PNGs into `docs/screen-`

shots/ , overwriting what was there. It also listens for page errors and reports any screenshot whose body came out empty — so it doubles as a crude end-to-end check.

Note how it fills the login form: `input[name="phone_number"]` . That name exists only because `nameFromLabel()` derives it from the label text "Phone number". Rename that label and this script breaks.

43. Troubleshooting

Symptom	Cause	Fix
"Could not reach the server. Is Django running on http://127.0.0.1:8001?"	Backend down, or on another port	<code>python manage.py runserver 8001</code> — note the port
CORS error in the browser console	Your origin is not on the allow list	Add it to <code>CORS_ALLOWED_ORIGINS</code> and re-start Django
Access denied for user 'lms_user'@'localhost'	The grant was made for the wrong host	Grant to 'lms_user'@'127.0.0.1' — MySQL treats the two as different
<code>pip install mysqlclient</code> fails	Missing MySQL client headers	<code>brew install mysql-client pkg-config</code> / <code>apt install libmysqlclient-dev</code>
<code>makemigrations</code> wants to rewrite every primary key	<code>DEFAULT_AUTO_FIELD</code> was removed or changed	Put <code>BigAutoField</code> back; the columns are already <code>bigint</code>
"Only an admin can create accounts."	Signed in as a teacher or student	Use an admin, or <code>createsuperuser</code>
Signed in but every page is empty and the console shows 401	The token expired (8 hours)	Log in again. There is no refresh endpoint.
Reset email never arrives	The dev backend prints instead of sending	Copy the link out of the <code>runserver</code> terminal
Signed out straight after changing your password	Deliberate	Log in with the new password
Vite says the port is taken	5180 is busy	It picks the next free port and prints it — add that origin to CORS
A dropdown is empty but the table has rows	The lookup <code>listAll()</code> call failed	Check the network tab; the error also lands in the page's red banner
Grading a submission returns 400	It already has a result — <code>Results.submission</code> is one-to-one	Edit the existing result instead. The dropdown only offers ungraded ones.
An admin sees the dashboard instead of <code>/accounts</code>	The role had not arrived yet on a restored session	Already handled by <code>ProtectedRoute</code> ; if it persists, check <code>/profile/</code> returns a role
The smoke test exits 2 with "Could not sign in"	The demo accounts do not exist	<code>python manage.py seed_data</code>

44. Before it faces the real world

This is a development configuration and does not pretend otherwise. Here is the full list of what would have to change, in the order it matters.

#	Change	Where
1	Move <code>SECRET_KEY</code> to the environment and rotate it — the committed one is in git history	<code>lms/settings.py</code>
2	<code>DEBUG = False</code> , and set <code>ALLOWED_HOSTS</code> to the real hostname	<code>lms/settings.py</code>
3	Move the database password out of the defaults — <code>os.getenv</code> is already there, the fallback is the problem	<code>lms/settings.py</code>
4	Delete every seeded account, especially the three demo logins with published passwords	database
5	Shorten <code>ACCESS_TOKEN_LIFETIME</code> and add a refresh endpoint, so a token can actually be revoked	<code>settings.py</code> , <code>urls.py</code> , <code>api.js</code>
6	Set <code>CORS_ALLOWED_ORIGINS</code> to the real frontend origin only	<code>lms/settings.py</code>
7	Serve over HTTPS, and set <code>SECURE_SSL_REDIRECT</code> , <code>SESSION_COOKIE_SECURE</code> , <code>CSRF_COOKIE_SECURE</code> , <code>HSTS</code>	<code>lms/settings.py</code>
8	Real email: switch <code>EMAIL_BACKEND</code> to SMTP and supply credentials from the environment. Remove the hardcoded address in <code>EMAIL_HOST_USER</code>	<code>lms/settings.py</code>
9	Static files and a real server — <code>collectstatic</code> , gunicorn behind nginx. <code>runserver</code> is not for production	deployment
10	Rate-limit <code>/login/</code> and <code>/password-reset/</code> . Neither has any throttling today	DRF throttles
11	Reconsider the token in <code>localStorage</code> — readable by any script on the origin	<code>api.js</code>

WARNING

Items 1 and 4 are the ones that matter most and are easiest to forget. The secret key is already public in the repository's history, so rotating it is not optional — changing the file without changing the value achieves nothing.

45. Rough edges and known gaps

An honest inventory. None of these stop the app working; all of them are worth knowing before you build on it.

45.1 Structural

Gap	Consequence
Nothing links Student to User	The server cannot tell whose submission is whose. Hence the odd submission permission, and hence a student being able to <code>POST</code> a submission naming somebody else as its author.
No unique constraint on Enrollment	The same student can be enrolled on the same course twice. A <code>unique_together</code> would fix it.
Assignment has both a course and a lesson	Nothing checks that the lesson belongs to the course. The UI narrows the dropdown; the API does not care.
No Meta.ordering on any model	Harmless — every list view sets <code>ordering</code> explicitly — but a queryset used anywhere else has no defined order.
Teacher/Student rows and login accounts are joined by email convention only	Deleting one leaves the other. The seeder maintains the convention; nothing enforces it.

45.2 Authentication

Gap	Consequence
No token refresh endpoint	The refresh token is issued, handed over and discarded. The access token is widened to 8 hours to compensate.
No logout endpoint	Logging out is client-side only. The token stays valid until it expires.
Access tokens cannot be revoked	A password change blacklists refresh tokens only. See chapter 35 .
A superuser cannot use the app's login page	No <code>Profile</code> , so no phone number to sign in with. <code>/admin/</code> only.
No rate limiting anywhere	<code>/login/</code> can be hammered.

45.3 Testing and tooling

Gap	Consequence
<code>tests.py</code> is empty	No unit tests. The smoke test is thorough but needs a running server and a seeded database, so it cannot run in a clean CI job unaided.
<code>requests</code> is not in <code>requirements.txt</code>	The smoke test's only dependency is undeclared.
No frontend tests	<code>npm run build</code> and the screenshot script are the whole safety net.
<code>no-unused-vars</code> ignores capitalised names	A genuinely unused component import is never reported.
No CI configuration	Nothing runs the lint or the smoke test automatically.

45.4 Small inconsistencies

- `DEMO_ACCOUNTS` is duplicated in `seed_data.py` and `Login.jsx`, and the two have drifted — the login page offers only the admin.
- `/submission/` is singular, `/results/` is plural.
- The model is `Results` (plural) but the serializer is `ResultSerializer` (singular).
- The detail views have no `select_related` while their list views do.
- `backend/backend/urls.py` gives names to the resource routes but the two password-reset paths have none.
- `blacklist_user_tokens()` swallows every exception silently, so a broken blacklist app would never be noticed.
- The frontend never uses `PATCH` for resources — always `PUT` with the full row — even though the API supports both.
- There is a `frontend/dist/` directory on disk from an earlier build; it is gitignored and not tracked.

NOTE

None of the above is a bug in the sense of "this does not work". They are the places where the next person will be surprised, which is what this chapter is for.

APPENDICES

Reference

The API surface, every file with its size, and the vocabulary this codebase uses.

A. API reference

Base URL `http://127.0.0.1:8001/api`. Every endpoint except the three public ones requires `Authorization: Bearer <access token>`.

A.1 Accounts

Method	Path	Who	Body / result
POST	<code>/login/</code>	public	<code>{phone, password}</code> → <code>{message, user_id, username, role, tokens: {access, refresh}}</code>
POST	<code>/register/</code>	admin	<code>{username, email, password, phone, first_name, last_name?, role?}</code> → 201 with the created user
GET	<code>/profile/</code>	signed in	→ <code>{message, user: {user_id, username, email, first_name, last_name, phone, role}}</code>
PATCH	<code>/profile/</code>	signed in	<code>{first_name?, last_name?, email?, phone?}</code> . <code>role</code> and <code>username</code> are ignored.
POST	<code>/change-password/</code>	signed in	<code>{current_password, new_password, confirm_password}</code> → 200, and every refresh token is blacklisted
POST	<code>/password-reset/</code>	public	<code>{email}</code> → always 200
POST	<code>/password-reset-confirm/</code>	public	<code>{uid, token, new_password, confirm_password}</code>

A.2 Resources

Every one has the same two routes:

HTTP

```
GET /api/<resource>/      list, paginated and filtered
POST /api/<resource>/     create

GET /api/<resource>/<id>/  one row
PUT /api/<resource>/<id>/  replace
PATCH /api/<resource>/<id>/ partial update
DELETE /api/<resource>/<id>/ 204 on success
```

Resource	Path	Fields sent	Writable by
Teachers	/teacher/	id, name, email, subject, is_active	admin
Students	/student/	id, name, email, enrollment_date, is_active, roll_number	admin
Courses	/course/	id, title, description, teacher, <i>teacher_name</i>	admin, teacher
Enrollments	/enrollment/	id, student, course, enrollment_date, <i>student_name</i> , <i>course_title</i>	admin, teacher
Lessons	/lesson/	id, title, description, course, <i>course_title</i>	admin, teacher
Assignments	/assignment/	id, title, description, lesson, due_date, course, <i>course_title</i> , <i>lesson_title</i>	admin, teacher
Submissions	/submission/	id, assignment, student, submitted_at, content, <i>student_name</i> , <i>assignment_title</i>	admin, teacher; students may POST
Results	/results/	id, submission, score, feedback, <i>student_name</i> , <i>assignment_title</i>	admin, teacher

Italic fields are read-only and computed from a relation. `enrollment_date` on an enrollment and `submitted_at` on a submission are `auto_now_add` and cannot be supplied.

A.3 Query parameters

Accepted by every list route:

Parameter	Example	Meaning
search	?search=nusrat	Case-insensitive contains, OR'd over that view's <code>search_fields</code>
ordering	?ordering=-due_date	Sort. Leading <code>-</code> is descending. Must be on <code>ordering_fields</code> .
page	?page=3	Past the end → 404 "Invalid page"
page_size	?page_size=50	Default 10, capped at 200

Plus the filters each view declares:

Resource	Filter parameters	ORM lookup
Teachers	<code>is_active</code> , <code>subject</code>	<code>is_active</code> , <code>subject__icontains</code>
Students	<code>is_active</code> , <code>enrolled_from</code> , <code>enrolled_to</code>	<code>is_active</code> , <code>enrollment_date__gte/__lte</code>
Courses	<code>teacher</code>	<code>teacher</code>
Lessons	<code>course</code>	<code>course</code>
Enrollments	<code>student</code> , <code>course</code> , <code>enrolled_from</code> , <code>enrolled_to</code>	<code>student</code> , <code>course</code> , <code>enrollment_date__gte/__lte</code>
Assignments	<code>course</code> , <code>lesson</code> , <code>due_from</code> , <code>due_to</code>	<code>course</code> , <code>lesson</code> , <code>due_date__gte/__lte</code>
Submissions	<code>assignment</code> , <code>student</code> , <code>course</code> , <code>submitted_from</code> , <code>submitted_to</code> , <code>ungraded</code>	<code>...</code> , <code>assignment__course</code> , <code>submitted_at__gte/__lte</code> , <code>results__isnull</code>
Results	<code>submission</code> , <code>student</code> , <code>assignment</code> , <code>score_min</code> , <code>score_max</code>	<code>submission</code> , <code>submission__student</code> , <code>submission__assignment</code> , <code>score__gte/__lte</code>

A.4 The list envelope

JSON

GET /api/student/?search=nusrat&is_active=true&ordering=name&page=2

```
{
  "count": 214,
  "page": 2,
  "page_size": 10,
  "total_pages": 22,
  "next": "http://127.0.0.1:8001/api/student/?page=3&...",
  "previous": "http://127.0.0.1:8001/api/student/?page=1&...",
  "results": [ { "id": 11, "name": "Nusrat Jahan", "...": "..." } ]
}
```

A.5 Status codes you will see

Code	Means
200	Read, update, or a deliberately vague success (password reset request)
201	Created
204	Deleted — no body
400	Validation. Body is either <code>{"field": ["..."]}</code> or <code>{"error": "..."}</code>
401	No token, or a token the server will not accept
403	Valid token, wrong role. Body is <code>{"detail": "<the permission class's message>"}</code>
404	No such row, or a page past the end of a list

B. File index

Every source file, with its size, in the order this book covers them.

File	Lines	Covered in
backend/		
manage.py	22	13.1
requirements.txt	6	2.1
lms/settings.py	~190	13.4
lms/urls.py	22	13.2
lms/wsgi.py, lms/asgi.py	16 each	13.3
backend/models.py	~100	14
backend/serializers.py	~290	15
backend/permissions.py	~95	16
backend/filters.py	~135	17
backend/pagination.py	~40	18
backend/views.py	~380	19
backend/urls.py	~45	20
backend/admin.py	~75	21
backend/tests.py	3	45.3
backend/migrations/*.py	4 files	22
backend/seed_catalog.py	470	23.1
backend/management/commands/seed_data.py	488	23
scripts/api_smoke_test.py	487	24
frontend/src/		
main.jsx	14	25.2
index.css	1	25.3
api.js	~280	26
auth.js	15	27
AuthContext.jsx	~95	27
permissions.js	~45	28
useTableQuery.js	~135	29
App.jsx	~70	30
Sidebar.jsx	~290	31
components/ (13 files)	685	32
Pagination.jsx	134	32.5
Table.jsx	104	32.4
ConfirmDialog.jsx	80	32.2
Input.jsx	66	32.3
FilterBar.jsx	60	32.2

Alert.jsx	53	32.2
Select , ProtectedRoute , Textarea , Button , Checkbox , field.js , IconButton , index.js , PageHeader , styles.js	4-31	32
pages/ (15 files)	4,408	33, 34
Assignments.jsx	504	33.7
Submissions.jsx	493	33.7
Results.jsx	490	33.7
Students.jsx	421	33
Enrollments.jsx	414	33
Lessons.jsx	372	33
Courses.jsx	371	33
Teachers.jsx	367	33
Profile.jsx	242	34.6
Accounts.jsx	171	34.5
Login.jsx	154	34.2
Dashboard.jsx	143	34.1
ResetPassword.jsx	121	34.4
ForgotPassword.jsx	111	34.3
NotFound.jsx	34	34.7
docs/		
capture_screenshots.py	~180	42.4

C. Glossary

Term	In this project it means
the app	<code>backend/backend/</code> — the single Django app. Not the React application, and not the <code>lms/</code> project package.
role	One of <code>admin</code> , <code>teacher</code> , <code>student</code> , stored on <code>Profile.role</code> . A superuser counts as admin without having one.
the envelope	The <code>{count, page, page_size, total_pages, next, previous, results}</code> object every list endpoint returns.
<code>filter_fields</code>	A view attribute: <code>{query parameter: ORM lookup}</code> . Read by <code>FieldFilterBackend</code> . Undeclared parameters are ignored.
<code>listAll</code>	The <code>api.js</code> call that walks every page and returns just the rows. For dropdowns, never for tables.
lookup list	A full list of one resource held in a page's state purely to populate a <code><select></code> .
the six steps	The shape every resource page follows: state, load effect, reload, save, delete, JSX.
<code>dice()</code>	The seeder's stable pseudo-random helper — <code>crc32</code> of the inputs, mod 100. What makes re-seeding a no-op.
demo accounts	The three seeded logins on <code>01700000000-2</code> . Repaired on every seed run.
the mirror	<code>frontend/src/permissions.js</code> — a copy of the backend rules used only to decide which buttons to draw.
ungraded	A submission with no <code>Results</code> row pointing at it. Expressed as <code>?ungraded=true → results__isnull=True</code> .
<code>stepBack-After</code>	The hook helper that turns a 404 "Invalid page" into a move to the previous page.
the smoke test	<code>backend/scripts/api_smoke_test.py</code> . Not a unit test suite — it needs a running server and seeded data.

NOTE

Regenerating this book. The source is [docs/guidebook.html](https://docs.guidebook.html). Rebuild the PDF with:

SHELL

```
python3 -m venv /tmp/pdfenv && /tmp/pdfenv/bin/pip install weasyprint
/tmp/pdfenv/bin/weasyprint docs/guidebook.html docs/LMS-Guidebook.pdf
```

Screenshots are referenced relatively, so the HTML must stay in `docs/`.